

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE – CARRERA DE INGENIERÍA DE SOFTWARE

Asignatura: Desarrollo de Aplicaciones Móviles
Proyecto Final



Sigchos Agrotech Detección de Enfermedades en Hojas de Zapallo mediante Inteligencia Artificial

Integrantes:

Alfonso Arroyo
Pedro Frías
Ariel Llumiquinga
Germán Cáceres

Docente: Ing. Doris Chicaiza MSc

Fecha: 14 de julio de 2026

Repositorio del proyecto:

<https://github.com/alfonso1031/sigchos-agrotech>

Índice

Índice de Algoritmos	iii
Índice de Anexos	iv
Tema	1
1 Descripción del Sistema	1
1.1 Descripción	1
1.2 Problema que resuelve	1
1.3 Funcionalidades técnicas	2
1.4 Hardware y servicios utilizados	3
1.5 Tecnologías	3
2 Introducción	3
3 Planteamiento del Problema	4
3.1 Objetivo General	4
3.2 Objetivos Específicos	5
4 Alcance	5
5 Arquitectura del Sistema	6
5.1 Visión general	6
5.2 Clean Architecture y MVVM	6
5.3 Flujo del caso de uso central: diagnóstico de una hoja	8
6 Desarrollo	8
6.1 Conceptos	8
6.1.1 El cultivo de zapallo	8
6.1.2 Enfermedades foliares consideradas	9
6.1.3 Transferencia de aprendizaje y MobileNetV2	9
6.1.4 Inferencia <i>on-device</i> con TensorFlow Lite	9
6.1.5 Origen y preparación de los datos de entrenamiento	10
6.1.6 Hiperparámetros y proceso de entrenamiento	10
6.1.7 Métricas del modelo	11
6.2 Desarrollo del Programa	11
6.2.1 Capa <i>domain</i> : entidades y casos de uso	11
6.2.2 Composición de dependencias en <code>main.dart</code>	12
6.2.3 Capa <i>presentation</i> : ViewModel reactivo	12
6.2.4 Capa <i>data</i> : servicio de inferencia TFLite	14
6.2.5 Tolerancia a fallo en la subida de imágenes	15
6.2.6 Regla de riesgo fúngico (clima)	16
6.2.7 Validación de cédula ecuatoriana (módulo 10)	16
6.2.8 Polígono de finca y cálculo de área	17
6.2.9 Gestión de permisos con aviso previo	18
6.2.10 Protección de claves de API	18
7 Panel Web Administrativo	18

7.1	Control de acceso	19
7.2	Arquitectura del panel	19
7.3	Vistas del panel	20
7.3.1	Resumen (<i>dashboard</i>)	20
7.3.2	Diagnósticos	21
7.3.3	Agricultores	21
7.3.4	Modelo IA	21
7.4	Despliegue	22
8	Pruebas	22
8.1	Estrategia general	22
8.2	Pruebas de usuario — usabilidad	22
8.2.1	Protocolo	22
8.2.2	Aspectos de usabilidad ya incorporados al diseño	23
8.3	Pruebas de rendimiento — pruebas del sistema	24
8.4	Casos de prueba funcionales (pruebas de sistema)	24
8.5	Evidencia de pruebas	27
9	Diagrama de Carpetas	27
10	Modelo de Datos	29
10.1	Colecciones	29
10.2	Relaciones	30
10.3	Índices compuestos	30
10.4	Reglas de seguridad	30
11	Trabajos Relacionados	31
12	Avance Técnico	33
12.1	Arquitectura aplicada	33
12.2	Estructura del proyecto	33
12.3	Pantallas desarrolladas	34
12.4	Código principal explicado	35
12.5	Base de datos utilizada	35
12.6	Evidencia de funcionamiento	35
12.7	Problemas encontrados	36
12.8	Mejoras pendientes	37
13	Conclusiones	37
14	Recomendaciones	38
	Referencias	40
	Anexos	41

Índice de figuras

1	Arquitectura general del sistema Sigchos Agrotech	6
2	Clean Architecture: dirección de dependencias por <i>feature</i>	7

3	Flujo MVVM + Provider a través de las capas de Clean Architecture .	7
4	Flujo del diagnóstico de una hoja (Captura → Analizando → Resultado)	8
5	Arquitectura del panel web administrativo (MVVM simplificado)	20
6	Relaciones lógicas entre colecciones (referencias por ID, sin <i>join</i> nativo)	30

Índice de cuadros

1	Funcionalidades técnicas implementadas	2
2	Stack tecnológico	3
3	Catálogo de clases foliares implementado	9
4	Configuración de entrenamiento del modelo (Google Colab, GPU T4) .	10
5	Precisión y <i>recall</i> por clase (conjunto de validación)	11
6	Casos de prueba de usabilidad	23
7	Casos de prueba de rendimiento	24
8	Casos de prueba funcionales del sistema	24
9	Colecciones de Firestore	29
10	Índices compuestos declarados	30
11	Pantallas implementadas en la app móvil (mínimo requerido: 4)	34
12	Pantallas implementadas en el panel web administrativo	35
13	Inventario de capturas para la carpeta <code>img/</code>	41

Índice de Algoritmos

1	Clasificación de una hoja de zapallo	15
2	Validación de cédula ecuatoriana (módulo 10)	17

Índice de Anexos

1	Pantalla de Login	43
2	Registro con validación de cédula	43
3	Pantalla de Inicio	43
4	Aviso previo de permiso de cámara	43
5	Captura de hoja con cámara en vivo	43
6	Resultado del diagnóstico	43
7	Rechazo de imagen que no es hoja (clase <code>no_hoja</code>)	43
8	Recomendaciones agrícolas	43
9	Historial de diagnósticos filtrable	43
10	Mapa de zonas afectadas con polígono de finca	43
11	Clima y alerta de riesgo fúngico	43
12	Trazado del polígono de la finca con área calculada	43
13	Perfil del agricultor	43
14	Notificación local de resultado	43
15	Panel web administrativo — Resumen (KPIs, gráficos y alertas de zona)	44
16	Panel web administrativo — Diagnósticos globales filtrables	44
17	Panel web — Agricultores	44
18	Panel web — Modelo IA	44
19	Panel web — Login restringido a la colección <code>admins</code>	44

20	<code>flutter analyze</code> sin advertencias	45
21	Compilación exitosa del APK	45
22	Documento de <code>diagnosticos_hojas</code> en Firestore	45
23	Imágenes de hojas en Firebase Storage	45
24	Entrenamiento del modelo en Google Colab	45
25	Matriz de confusión sobre el conjunto de validación	45
26	Aplicación distribuida en el canal de prueba interna de Google Play . .	45

Tema

Sigchos Agrotech: Detección de Enfermedades en Hojas de Zapallo mediante Inteligencia Artificial. Aplicación móvil Android que permite a los agricultores del cantón Sigchos (provincia de Cotopaxi, Ecuador) fotografiar hojas de zapallo y obtener un diagnóstico fitosanitario preliminar mediante un modelo de aprendizaje profundo ejecutado directamente en el dispositivo, con apoyo de geolocalización, datos climáticos, recomendaciones agronómicas y un panel web administrativo de monitoreo agregado.

1. Descripción del Sistema

1.1 Descripción

Sigchos Agrotech es una aplicación móvil inteligente que detecta enfermedades en hojas de zapallo mediante la captura de imágenes con la cámara del celular. El sistema analiza la imagen con un modelo de inteligencia artificial que se ejecuta **en el propio dispositivo** (sin enviar la foto a un servidor de inferencia), muestra el diagnóstico preliminar con su porcentaje de confianza y genera recomendaciones de manejo agrícola para el agricultor. Cada diagnóstico queda geoetiquetado y asociado al clima del momento, lo que habilita un mapa de incidencia por finca y alertas de riesgo fúngico. El sistema se complementa con un **panel web administrativo** que permite a un técnico o supervisor monitorear de forma agregada todos los diagnósticos, los agricultores registrados y el rendimiento del clasificador.

1.2 Problema que resuelve

Los agricultores identifican las enfermedades foliares de forma visual y empírica, lo que puede generar diagnósticos tardíos o equivocados y afectar el crecimiento y rendimiento de la planta.

1.3 Funcionalidades técnicas

Cuadro 1: Funcionalidades técnicas implementadas

Funcionalidad	Implementación
Autenticación	Firestore Authentication: correo/contraseña y Google Sign-In.
Gestión jerárquica	CRUD de fincas → parcelas → cultivos, cada una con GPS.
Polígono de finca	Trazado del contorno real de la finca tocando el mapa; área calculada con fórmula esférica.
Captura de hoja	Cámara <i>en vivo</i> embebida (CameraController) con marco guía, o selección desde galería.
Clasificación IA	TensorFlow Lite <i>on-device</i> , MobileNetV2 por transferencia de aprendizaje, 6 clases (5 enfermedades + rechazo no_hoja).
Rechazo de no-hojas	Clase entrenada no_hoja + umbral mínimo de confianza (0,50).
Geoetiquetado	geolocator : latitud/longitud guardadas como GeoPoint en cada diagnóstico.
Snapshot climático	Temperatura, humedad y descripción del clima embebidos en el documento del diagnóstico.
Mapa de incidencia	google_maps_flutter en modo satelital, polígonos de finca, filtros con/sin daño, leyenda de severidad.
Clima y pronóstico	API REST de OpenWeather: clima actual + pronóstico de 5 días.
Alerta de riesgo fúngico	Regla: humedad $\geq 80\%$ y temperatura entre 15–25 °C.
Notificaciones locales	flutter_local_notifications : resultado de diagnóstico, alerta climática y recordatorio semanal.
Recomendaciones	Fichas por enfermedad desde Firestore, con contenido de respaldo local si la colección no está poblada.
Historial	Lista filtrable por clase, con eliminación en cascada (documento + imagen en Storage).
Tolerancia a fallo	Subida a Storage con 3 reintentos y <i>backoff</i> exponencial; si falla, el diagnóstico se guarda igual.
Validaciones	Cédula ecuatoriana (módulo 10), celular (prefijo 09, 10 dígitos), correo, campos obligatorios, fechas no futuras.
Permisos	Diálogo de aviso previo antes de solicitar cámara/GPS y manejo de <i>denegado permanentemente</i> .
Panel web	Flutter Web con control de acceso por colección admins y cuatro vistas de monitoreo.

1.4 Hardware y servicios utilizados

- **Cámara** del dispositivo (paquete `camera`, vista previa en vivo dentro de la app; `image_picker` para galería).
- **GPS** (paquete `geolocator`, precisión alta).
- **API de clima**: OpenWeather (endpoints `/weather` y `/forecast`).
- **Firebase**: Authentication, Cloud Firestore, Storage y Hosting.
- **TensorFlow Lite**: inferencia del clasificador en el dispositivo.
- **Google Maps**: SDK for Android (mapa satelital, polígonos y marcadores).

1.5 Tecnologías

Cuadro 2: Stack tecnológico

Categoría	Tecnología / paquete
Framework	Flutter (Dart 3.x)
Arquitectura	Clean Architecture (<i>domain / data / presentation</i>)
Patrón de presentación	MVVM
Gestión de estado	Provider (<code>ChangeNotifier</code> , <code>MultiProvider</code>)
Autenticación	<code>firebase_auth</code> , <code>google_sign_in</code>
Base de datos	<code>cloud_firestore</code>
Almacenamiento	<code>firebase_storage</code>
IA <i>on-device</i>	<code>tflite_flutter</code> , <code>image</code>
API de clima	OpenWeather vía <code>http</code>
Geolocalización	<code>geolocator</code>
Cámara / galería	<code>camera</code> , <code>image_picker</code>
Mapas	<code>google_maps_flutter</code>
Permisos	<code>permission_handler</code>
Notificaciones	<code>flutter_local_notifications</code>
Gráficos	<code>fl_chart</code>
Panel web	Flutter Web + Firebase Hosting

2. Introducción

El cultivo de zapallo (*Cucurbita* spp.) constituye una actividad económica relevante para pequeños y medianos agricultores del cantón Sigchos, en la provincia de Cotopaxi. Sin embargo, este cultivo es susceptible a diversas enfermedades foliares de origen fúngico, viral y por deficiencia nutricional que, al no ser detectadas oportunamente, reducen significativamente el rendimiento y la calidad de la cosecha.

Tradicionalmente, el diagnóstico de estas enfermedades depende de la inspección visual del agricultor o de la visita de un técnico agrónomo: un proceso lento, subjetivo y no siempre disponible en zonas rurales. El avance de la inteligencia artificial aplicada a visión por computadora, junto con la masificación de los teléfonos inteligentes, permite

hoy trasladar capacidades de diagnóstico especializado directamente a la mano del agricultor. La literatura reciente demuestra que las redes neuronales convolucionales alcanzan exactitudes competitivas en la clasificación de enfermedades foliares [5, 1] y que su despliegue en dispositivos móviles mediante TensorFlow Lite es técnicamente viable incluso sin conexión a internet [3, 4].

El presente documento constituye el **informe final del Proyecto Final** de la asignatura *Desarrollo de Aplicaciones Móviles*. En él se documenta **Sigchos Agrotech**: una aplicación móvil Android construida en **Flutter**, que aplica **Clean Architecture** y el patrón **MVVM** con **Provider** como gestor de estado, integra un modelo de **TensorFlow Lite** entrenado mediante transferencia de aprendizaje sobre **MobileNetV2**, consume la **API REST de OpenWeather**, utiliza hardware móvil (cámara, GPS y Google Maps) y persiste su información en **Firebase** (Authentication, Firestore y Storage). El proyecto se complementa con un **panel web administrativo** (Flutter Web) para el monitoreo de la incidencia fitosanitaria a nivel agregado.

3. Planteamiento del Problema

Los agricultores del cantón Sigchos identifican las enfermedades foliares del zapallo principalmente de forma visual y empírica, sin herramientas de apoyo diagnóstico. Esta práctica genera cuatro problemas concretos:

- **Diagnóstico tardío**: los síntomas iniciales de enfermedades como el mildiú vellosa o el oídio suelen confundirse entre sí o pasar desapercibidos, permitiendo que la enfermedad avance antes de aplicar un control.
- **Uso inadecuado de agroquímicos**: al no identificar con precisión el patógeno, el agricultor puede aplicar fungicidas incorrectos, incrementando costos y afectando el ambiente.
- **Falta de trazabilidad y monitoreo zonal**: no existe un registro digital de dónde y cuándo ocurren los brotes, lo que impide anticipar focos de propagación entre fincas cercanas.
- **Ausencia de correlación climática**: la humedad relativa y la temperatura son factores determinantes en la proliferación de hongos foliares, pero el agricultor no dispone de alertas automáticas que relacionen el clima local con el riesgo fitosanitario.

A esto se suma una restricción de contexto: la conectividad móvil en las zonas rurales del cantón es intermitente, por lo que una solución que dependa de enviar cada fotografía a un servidor de inferencia remoto resulta poco confiable en campo.

En consecuencia, se plantea la siguiente pregunta de investigación: *¿Es posible desarrollar una aplicación móvil que, mediante inteligencia artificial embebida y datos de contexto (geolocalización y clima), permita a un agricultor de Sigchos obtener un diagnóstico preliminar confiable de enfermedades foliares en zapallo y recomendaciones de manejo agrícola en el momento y lugar donde se encuentra?*

3.1 Objetivo General

Desarrollar una aplicación móvil para Android, estructurada bajo Clean Architecture y el patrón MVVM con Provider, que permita a los agricultores del cantón Sigchos detectar tempranamente enfermedades foliares en cultivos de zapallo mediante la clasificación de imágenes con un modelo de inteligencia artificial ejecutado en el propio dispositivo,

integrando geolocalización y datos climáticos para generar recomendaciones agrícolas contextualizadas, y complementada con un panel web administrativo para el monitoreo agregado de la incidencia fitosanitaria.

3.2 Objetivos Específicos

1. **Diseñar** la arquitectura de la aplicación móvil aplicando **Clean Architecture** (capas *domain*, *data* y *presentation*) y el patrón **MVVM** con **Provider** como gestor de estado reactivo, garantizando una regla de dependencia unidireccional en todas las *features* del proyecto.
2. **Entrenar** un modelo de clasificación de enfermedades foliares del zapallo mediante transferencia de aprendizaje sobre **MobileNetV2** y **desplegarlo** en formato **TensorFlow Lite** para realizar inferencia sin conexión directamente en el dispositivo móvil, incorporando una clase de rechazo que descarte imágenes que no correspondan a una hoja de zapallo.
3. **Integrar** los servicios de **Firebase Authentication**, **Cloud Firestore** y **Firebase Storage** para autenticar a los agricultores y persistir de forma segura fincas, parcelas, cultivos, diagnósticos e imágenes, aplicando reglas de seguridad basadas en la propiedad del documento.
4. **Consumir** la **API REST de OpenWeather** para obtener el clima actual y el pronóstico según la ubicación GPS del agricultor, y **generar** alertas automáticas de riesgo fúngico a partir de las condiciones de humedad y temperatura registradas.
5. **Incorporar** el uso de **hardware móvil** (cámara en vivo, GPS y Google Maps) para la captura georreferenciada de los diagnósticos y su visualización en un mapa de incidencia con el contorno real de cada finca.
6. **Desarrollar** un **panel web administrativo** independiente en Flutter Web que permita monitorear de forma agregada los diagnósticos, los agricultores registrados y el rendimiento del modelo de IA, con acceso restringido y sin exponer dicha funcionalidad como un rol dentro de la aplicación móvil.
7. **Implementar** un conjunto de **validaciones de formulario** conformes a los formatos ecuatorianos (cédula mediante el algoritmo de módulo 10 y número celular con prefijo 09) junto con las validaciones generales de correo, campos obligatorios y valores numéricos.
8. **Definir y documentar** un plan de **pruebas de usabilidad y de rendimiento** con casos de prueba trazables que permitan verificar el cumplimiento funcional y no funcional del sistema.

4. Alcance

El proyecto comprende dos aplicaciones independientes que comparten un mismo proyecto de Firebase:

- **Aplicación móvil (Android):** registro e inicio de sesión (correo/contraseña y Google Sign-In); gestión de fincas, parcelas y cultivos de zapallo, incluyendo el trazado del polígono de la finca sobre el mapa; captura de hojas mediante cámara en vivo o galería; clasificación con IA *on-device*; historial de diagnósticos filtrable; mapa satelital de incidencia con filtros y navegación a focos; consulta de clima con alertas de riesgo fúngico; recomendaciones agrícolas por enfermedad; notificaciones locales; y edición de perfil con foto.

- **Panel web administrativo (Flutter Web):** acceso restringido a cuentas registradas en la colección `admins`, con vistas de Resumen (KPIs y gráficos), Diagnósticos globales, Agricultores registrados y Modelo de IA.

No forma parte del alcance actual:

- La clasificación de la clase “daño por plaga” mediante IA: no se dispone de un dataset público etiquetado adecuado, por lo que la clase fue retirada del catálogo, de los colores y de las recomendaciones de ambas aplicaciones.
- El CRUD completo de recomendaciones y la emisión de alertas manuales desde el panel web (el panel es, hoy, de solo lectura/monitoreo).
- Las notificaciones *push* desde servidor: las notificaciones actuales son locales y se disparan desde el propio dispositivo.
- La versión iOS de la aplicación móvil.

5. Arquitectura del Sistema

5.1 Visión general

El sistema se compone de un cliente móvil, un cliente web y un *backend* gestionado (Firebase), más un servicio externo de clima. La inferencia de IA no viaja a la nube: ocurre íntegramente dentro del teléfono.

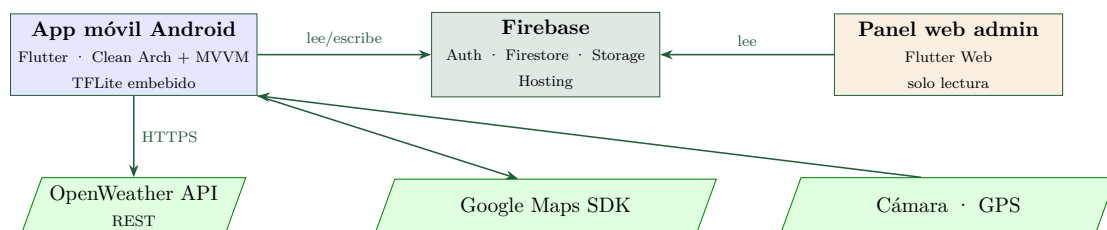


Figura 1: Arquitectura general del sistema Sigchos Agrotech

5.2 Clean Architecture y MVVM

Cada *feature* de la aplicación móvil (autenticación, fincas, parcelas, cultivos, diagnóstico, clima, historial, recomendaciones, mapa e inicio) se organiza en tres capas con una regla de dependencia unidireccional: la capa *presentation* depende de *domain*, y la capa *data* implementa las abstracciones definidas en *domain*. La capa *domain* no conoce Flutter ni Firebase, lo que la hace comprobable de forma aislada.

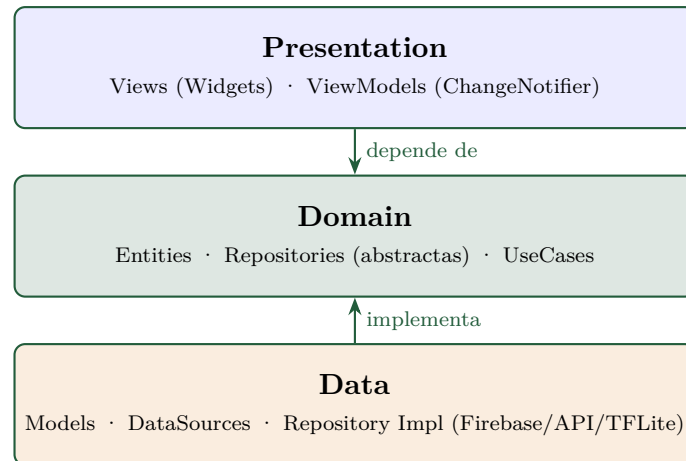
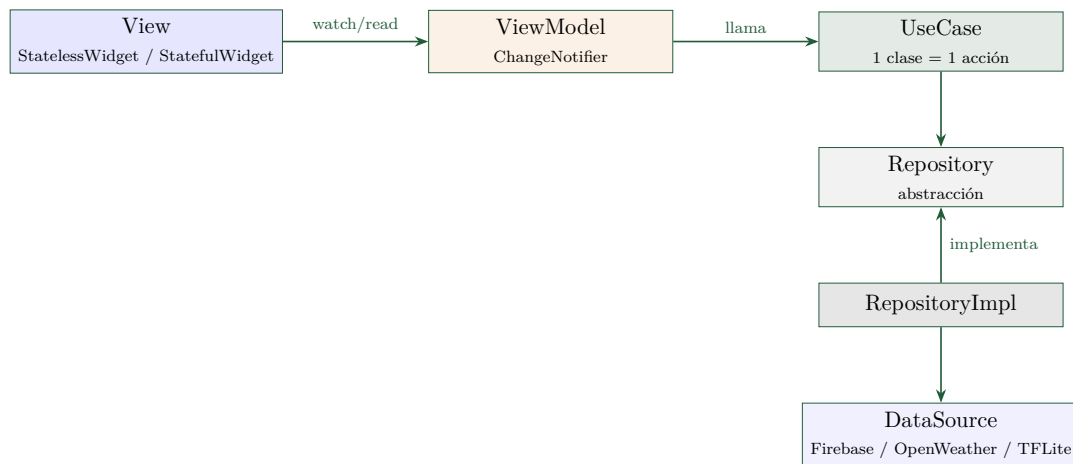
Figura 2: Clean Architecture: dirección de dependencias por *feature*

Figura 3: Flujo MVVM + Provider a través de las capas de Clean Architecture

Los `ViewModel` se registran globalmente en `main.dart` mediante un único `MultiProvider`, y cada Vista los consume con `context.watch<T>()` (para reconstruirse ante cambios) o `context.read<T>()` (para disparar acciones sin reconstruirse). La composición de dependencias es **manual**, sin *service locator*: cada `DataSource` se envuelve en su `RepositoryImpl`, este se inyecta en los `UseCase` y estos en el `ViewModel`.

5.3 Flujo del caso de uso central: diagnóstico de una hoja

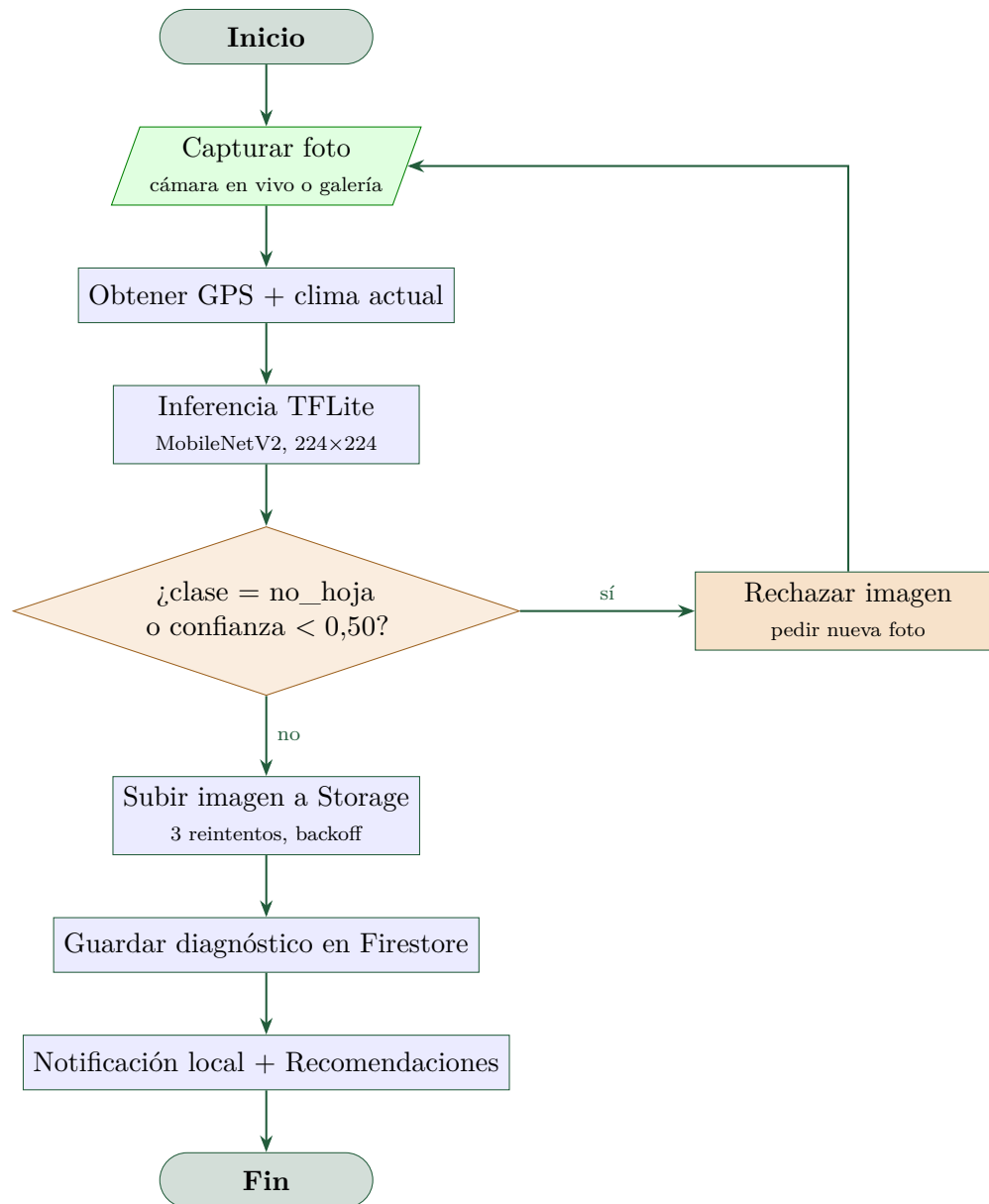


Figura 4: Flujo del diagnóstico de una hoja (Captura → Analizando → Resultado)

6. Desarrollo

6.1 Conceptos

6.1.1 El cultivo de zapallo

El zapallo pertenece al género *Cucurbita*, familia *Cucurbitaceae*. Es un cultivo de ciclo corto ampliamente sembrado en la sierra ecuatoriana, sensible a la humedad relativa alta y a la variación de temperatura, condiciones habituales en el clima húmedo-frío del cantón Sigchos.

6.1.2 Enfermedades foliares consideradas

El catálogo fitopatológico implementado en la aplicación contempla cinco clases de estado foliar, todas cubiertas por el modelo de clasificación, más una sexta clase técnica de rechazo (`no_hoja`) que no representa una enfermedad sino una imagen no válida.

Cuadro 3: Catálogo de clases foliares implementado

Clase	Nombre científico	Síntomas característicos
Hoja sana	Sin patógeno	Color verde uniforme, sin manchas, polvo ni perforaciones.
Mancha foliar	<i>Cercospora citrullina</i>	Manchas circulares marrones o grises con borde definido, a veces con halo amarillo.
Mildiú	<i>Pseudoperonospora cubensis</i>	Manchas angulares amarillas en el haz y moho grisáceo en el envés.
Oídio	<i>Podosphaera xanthii</i>	Polvo blanco harinoso sobre haz y envés; las hojas se amarillean y secan.
Amarillamiento	Deficiencia / virosis	Amarillamiento general o intermisional; posible transmisión por mosca blanca.
no_hoja*	—	Clase técnica de rechazo: objetos, fondos u hojas de otras especies.

*`no_hoja` no se muestra al usuario como diagnóstico: cuando el modelo la elige como clase ganadora, la aplicación rechaza la imagen y solicita una nueva captura.

6.1.3 Transferencia de aprendizaje y MobileNetV2

La *transferencia de aprendizaje* consiste en reutilizar una red convolucional preentrenada sobre un corpus grande y genérico (ImageNet) y reemplazar únicamente su capa de clasificación por una nueva, entrenada con el dominio específico. Este enfoque es el estándar en clasificación de enfermedades foliares porque permite alcanzar exactitudes altas con datasets de tamaño moderado [5]. Se eligió **MobileNetV2** por su bajo número de parámetros y su diseño orientado a dispositivos móviles, criterio coherente con la literatura sobre diagnóstico foliar *on-device* [2].

6.1.4 Inferencia on-device con TensorFlow Lite

TensorFlow Lite convierte un modelo entrenado a un formato binario optimizado (`.tflite`) que se ejecuta con un intérprete liviano en el teléfono. La ventaja operativa es determinante para el contexto rural del proyecto: el diagnóstico funciona **sin conexión a internet** y sin transmitir la imagen a un servidor, lo que reduce latencia, costo de datos y dependencia de cobertura [4].

6.1.5 Origen y preparación de los datos de entrenamiento

El modelo se entrenó con el conjunto de datos público *Pumpkin Leaf Diseases Dataset From Bangladesh* [7], compuesto por 2000 imágenes RGB distribuidas en cinco clases (400 imágenes por clase): hoja sana, mildiú vellosa, oídio, mosaico (mapeada a *amari-llamiento*) y mancha bacteriana (mapeada a *mancha foliar*). Este mismo dataset ha sido utilizado en trabajos académicos recientes sobre clasificación de enfermedades de zapallo con redes convolucionales [1, 2], lo que respalda su idoneidad.

Para la clase de rechazo `no_hoja` se construyó un subconjunto adicional con imágenes de hojas de otras especies y de objetos/fondos, dimensionado al promedio de imágenes de las clases de zapallo, y se aplicó `class_weight` balanceado para evitar el sesgo derivado del desbalance.

6.1.6 Hiperparámetros y proceso de entrenamiento

Cuadro 4: Configuración de entrenamiento del modelo (Google Colab, GPU T4)

Parámetro	Valor
Arquitectura base	MobileNetV2 preentrenada en ImageNet (<code>include_top=False</code>)
Tamaño de entrada	$224 \times 224 \times 3$
Aumento de datos	RandomFlip, rotación y zoom aleatorios
Cabeza de clasificación	GlobalAveragePooling2D $\rightarrow$ Dropout(0,3) $\rightarrow$ Dense(softmax)
Normalización	<code>preprocess_input</code> de MobileNetV2 como capa interna del modelo
Fase 1 (cabeza)	12 épocas, Adam, <i>learning rate</i> 1×10^{-3} , base congelada
Fase 2 (<i>fine-tuning</i>)	8 épocas, Adam, <i>learning rate</i> 1×10^{-5} , últimas 30 capas descongeladas
Tamaño de lote	32
Balanceo	<code>class_weight</code> calculado con <code>compute_class_weight('balanced')</code>
Exportación	TFLiteConverter con <code>Optimize.DEFAULT</code> y <code>float16</code>
Tamaño final del artefacto	4,3 MB (<code>assets/ml/model.tflite</code>)

Un detalle de implementación crítico: dado que `preprocess_input` quedó incorporado **dentro** del grafo del modelo, la aplicación debe alimentar al intérprete con los píxeles **crudos** en el rango [0, 255]. Normalizar también en Dart aplicaría la transformación dos veces y degradaría por completo la predicción; este error se detectó y corrigió durante la integración (véase la Sección 12.7).

6.1.7 Métricas del modelo

Sobre el conjunto de validación (400 imágenes, `classification_report` de *scikit-learn*), el clasificador alcanzó una **exactitud global del 83 %** y un **recall macro del 83 %**.

Cuadro 5: Precisión y *recall* por clase (conjunto de validación)

Clase	Precisión	Recall
Hoja sana	0,86	0,84
Mancha foliar	0,66	0,86
Mildiú	0,81	0,88
Oídio	0,96	0,72
Amarillamiento	0,95	0,85
Promedio macro	0,85	0,83

La clase más débil en precisión es *mancha foliar* (0,66): el modelo la confunde con mildiú y oídio, algo esperable dado que las tres son manchas foliares de origen fúngico con solapamiento visual en estadios tempranos. El *recall* más bajo corresponde a *oídio* (0,72). Ambas observaciones orientan las recomendaciones de la Sección 14.

6.2 Desarrollo del Programa

6.2.1 Capa domain: entidades y casos de uso

Las entidades son clases planas, sin dependencias de Flutter ni Firebase. El Listado 1 muestra `DiagnosticoEntity`, la entidad central del sistema.

```

1 class DiagnosticoEntity {
2   final String id;
3   final String usuarioId;
4   final String cultivoId;
5   final String imageUrl;
6   final String enfermedad;      // claseId ganador
7   final double confianza;        // 0..1
8   final List<ProbabilidadClase> top3;
9   final double? lat;
10  final double? lng;
11  final ClimateSnapshot? clima;  // clima embebido: evita un join
12                                   posterior
13  final DateTime fecha;
14
15  const DiagnosticoEntity({
16    required this.id, required this.usuarioId, required this.cultivoId
17    ,
18    required this.imageUrl, required this.enfermedad,
19    required this.confianza, required this.top3, required this.fecha,
20    this.lat, this.lng, this.clima,
21  });
22 }
```

Listing 1: Entidad de dominio del diagnóstico

Cada acción de negocio se modela como un caso de uso independiente (Listado 2), lo que facilita las pruebas unitarias y evita que el ViewModel conozca al repositorio concreto.

```

1 class ClasificarHojaUseCase {
2   final DiagnosticoRepository repository;
3   const ClasificarHojaUseCase(this.repository);
4
5   Future<List<ProbabilidadClase>> call(File imagen) =>
6     repository.clasificarHoja(imagen);
7 }

```

Listing 2: Caso de uso: clasificar una hoja

6.2.2 Composición de dependencias en main.dart

Todas las dependencias se construyen a mano siguiendo el sentido de la regla de Clean Architecture y se registran una única vez en un MultiProvider (Listado 3).

```

1 // Diagnóstico: IA + Storage + Firestore
2 final tfliteService = TFLiteService();
3 await tfliteService.cargarModelo();           // se carga una sola vez
4   al arrancar
5
6 final diagnosticoRepository = DiagnosticoRepositoryImpl(
7   dataSource: DiagnosticoFirestoreDataSource(),
8   tfliteService: tfliteService,
9   storageService: StorageService(),
10 );
11
12 runApp(
13   MultiProvider(
14     providers: [
15       ChangeNotifierProvider.value(value: authViewModel),
16       ChangeNotifierProvider(
17         create: (_) => DiagnosticoViewModel(
18           clasificarHojaUseCase: ClasificarHojaUseCase(
19             diagnosticoRepository),
20           crearDiagnosticoUseCase: CrearDiagnosticoUseCase(
21             diagnosticoRepository),
22           notificationService: notificationService,
23         ),
24       ),
25       // ... fincas, parcelas, cultivos, clima, historial, mapa,
26       // recomendaciones
27     ],
28     child: SigchosApp(rutaInicial: rutaInicial),
29   ),
30 );

```

Listing 3: Composición manual de dependencias (fragmento de main.dart)

6.2.3 Capa presentation: ViewModel reactivo

El DiagnosticoViewModel orquesta cámara, IA y persistencia. El estado se expone mediante propiedades públicas y notifyListeners(). La ubicación llega ya resuelta

desde la Vista (que gestionó el permiso con un aviso previo), manteniendo al ViewModel independiente del ciclo de vida de los permisos del sistema operativo.

```

1 Future<bool> analizarYGuardar({
2     required String usuarioId,
3     required String cultivoId,
4     ClimaSnapshot? clima,
5     double? lat,
6     double? lng,
7 }) async {
8     if (imagen == null) return false;
9     estado = DiagnosticoEstado.analizando;
10    notifyListeners();
11
12    try {
13        final probabilidades = await _clasificarHojaUseCase(imagen!);
14        final ganadora = probabilidades.first;
15
16        // Rechazo de imágenes que no son hoja de zapallo: lo decide sobre
17        // todo la
18        // clase entrenada 'no_hoja'; el umbral es solo un piso de
19        // seguridad.
20        if (ganadora.claseId == 'no_hoja' ||
21            ganadora.probabilidad < _umbralConfianzaMinima) { // 0.50
22            errorMessage = 'La imagen no parece una hoja de zapallo o no se
23            ve con
24            claridad. Acerca la cámara a una sola hoja, con buena luz.'
25            ;
26            estado = DiagnosticoEstado.esperandoFoto;
27            return false;
28        }
29
30        final entidad = DiagnosticoEntity(
31            id: '', usuarioId: usuarioId, cultivoId: cultivoId, imagenUrl: '
32            ',
33            enfermedad: ganadora.claseId, confianza: ganadora.probabilidad,
34            top3: probabilidades.skip(1).take(3).toList(),
35            fecha: DateTime.now(), lat: lat, lng: lng, clima: clima,
36        );
37
38        resultado = await _crearDiagnosticoUseCase(
39            diagnostico: entidad, imagen: imagen!,
40        );
41        estado = DiagnosticoEstado.listo;
42
43        if (ganadora.claseId != 'hoja_sana') {
44            await _notificationService?.resultadoDiagnostico(
45                enfermedad: infoDe(ganadora.claseId).nombre,
46                confianzaPorcentaje: (ganadora.probabilidad * 100).round(),
47            );
48        }
49        return true;
50    } on Failure catch (f) {
51        errorMessage = f.message;
52        estado = DiagnosticoEstado.esperandoFoto;
53        return false;
54    } finally {
55        notifyListeners();
56    }
57 }

```

```

51 }
52 }

```

Listing 4: Fragmento del ViewModel de diagnóstico: análisis, rechazo y guardado

6.2.4 Capa data: servicio de inferencia TFLite

El TFLiteService carga el intérprete y las etiquetas una sola vez al arrancar la aplicación. Si el asset no existe o el intérprete no puede inicializarse, el servicio cae en un **modo simulado** que permite seguir demostrando el flujo completo de la aplicación durante el desarrollo (Listado 5).

```

1 class TFLiteService {
2     static const int inputSize = 224;
3     Interpreter? _interpreter;
4     List<String>? _labels;
5     bool modeloCargado = false;
6
7     Future<void> cargarModelo() async {
8         try {
9             _interpreter = await Interpreter.fromAsset('assets/ml/model.
10                tflite');
11             final raw = await rootBundle.loadString('assets/ml/labels.txt');
12             _labels = raw.split('\n').map((e) => e.trim())
13                 .where((e) => e.isNotEmpty).toList();
14             modeloCargado = true;
15         } catch (e) {
16             modeloCargado = false; // cae a modo simulado, no rompe el
17                                     flujo
18             debugPrint('[TFLite] FALLO al cargar modelo -> modo SIMULADO: $e');
19         }
20     }
21
22     Future<List<ClasificacionHoja>> clasificar(File imagen) async {
23         return modeloCargado && _interpreter != null
24             ? _clasificarConModelo(imagen)
25             : _clasificarSimulado();
26     }
27 }

```

Listing 5: Carga tolerante a fallo del modelo TFLite

El preprocesamiento de la imagen (Listado 6) redimensiona a 224×224 y entrega los píxeles **sin normalizar**, porque la normalización ya vive dentro del grafo del modelo.

```

1 Future<List<ClasificacionHoja>> _clasificarConModelo(File imagen)
2     async {
3     final decodificada = img.decodeImage(await imagen.readAsBytes());
4     if (decodificada == null) return _clasificarSimulado();
5
6     final redimensionada = img.copyResize(
7         decodificada, width: inputSize, height: inputSize,
8     );
9
10    // El modelo TFLite incluye 'preprocess_input' de MobileNetV2 como
11    capa

```

```

10 // interna: aquí se pasan los píxeles CRUDOS [0,255]. Normalizar
    también acá
11 // aplicaría la transformación dos veces y produciría basura.
12 final input = List.generate(1, (_) => List.generate(inputSize,
13     (y) => List.generate(inputSize, (x) {
14         final p = redimensionada.getPixel(x, y);
15         return [p.r.toDouble(), p.g.toDouble(), p.b.toDouble()];
16     }
17 ));
18
19 final labels = _labels ?? _ordenClases;
20 final output = [List.filled(labels.length, 0.0)];
21 _interpreter!.run(input, output);
22
23 final resultados = <ClasificacionHoja>[
24     for (var i = 0; i < labels.length; i++)
25         ClasificacionHoja(labels[i], output[0][i]),
26 ];
27 resultados.sort((a, b) => b.probabilidad.compareTo(a.probabilidad));
28 return resultados;
29 }

```

Listing 6: Preprocesamiento e inferencia

Algoritmo 1 Clasificación de una hoja de zapallo**Require:** Imagen I capturada por el usuario**Ensure:** Diagnóstico persistido o rechazo de la imagen

```

1:  $I' \leftarrow \text{redimensionar}(I, 224 \times 224)$ 
2:  $T \leftarrow \text{tensor}(I')$  ▷ píxeles crudos [0, 255]
3:  $P \leftarrow \text{interprete.run}(T)$  ▷ vector de 6 probabilidades
4:  $P \leftarrow \text{ordenarDescendente}(P)$ 
5:  $g \leftarrow P[0]$  ▷ clase ganadora
6: if  $g.clase = \text{no\_hoja}$  or  $g.prob < 0,50$  then
7:     rechazar imagen y solicitar nueva captura
8:     return
9: end if
10:  $u \leftarrow \text{subirImagenConReintentos}(I, 3)$  ▷ cadena vacía si falla
11:  $d \leftarrow \text{DiagnosticoEntity}(g.clase, g.prob, P[1..3], gps, clima, u)$ 
12:  $\text{Firestore.guardar}(d)$ 
13: if  $g.clase \neq \text{hoja\_sana}$  then
14:      $\text{notificarUsuario}(g)$ 
15: end if

```

6.2.5 Tolerancia a fallo en la subida de imágenes

La subida a Firebase Storage reintentará ante fallos transitorios de red con *backoff* exponencial. Si tras los reintentos sigue fallando, el diagnóstico se guarda igualmente sin imagen en la nube: una falla de infraestructura no debe bloquear la funcionalidad principal del sistema (Listado 7).

```

1 Future<String> _subirConReintentos(
2     File imagen, String usuarioId, {int intentos = 3},
3 ) async {

```

```

4   for (var i = 0; i < intentos; i++) {
5       try {
6           return await storageService.subirImagenHoja(imagen, usuarioId);
7       } catch (_) {
8           if (i == intentos - 1) return '';           // tolerante a fallo
9           await Future.delayed(Duration(milliseconds: 800 * (1 << i)));
10      }
11  }
12  return '';
13 }

```

Listing 7: Subida con reintentos y *backoff* exponencial

6.2.6 Regla de riesgo fúngico (clima)

La entidad de clima encapsula la regla de negocio que dispara la alerta: humedad relativa igual o mayor al 80 % y temperatura entre 15 y 25 °C, rango en el que proliferan el mildiú veloso y la mancha foliar (Listado 8).

```

1  class ClimaActualEntity {
2      final String ciudad;
3      final double temperatura;
4      final int humedad;
5      final String descripcion;
6      final double vientoKmh;
7      final int probabilidadLluvia;
8
9      /// Riesgo fúngico alto: humedad >= 80% y temperatura entre 15-25 °C
10
11      bool get riesgoFungicoAlto =>
12          humedad >= 80 && temperatura >= 15 && temperatura <= 25;
13  }

```

Listing 8: Regla de alerta de riesgo fúngico

6.2.7 Validación de cédula ecuatoriana (módulo 10)

```

1  static String? cedulaEcuatoriana(String? value) {
2      if (value == null || value.trim().isEmpty) return 'Ingresa tu cédula';
3
4      final cedula = value.trim();
5      if (!RegExp(r'^\d{10}$').hasMatch(cedula)) return 'Debe tener 10 dígitos';
6
7      final provincia = int.parse(cedula.substring(0, 2));
8      if (provincia < 1 || provincia > 24) return 'Cédula no válida';
9
10     final tercerDigito = int.parse(cedula[2]);
11     if (tercerDigito > 6) return 'Cédula no válida';
12
13     final coeficientes = [2, 1, 2, 1, 2, 1, 2, 1, 2];
14     var suma = 0;
15     for (var i = 0; i < 9; i++) {
16         var valor = int.parse(cedula[i]) * coeficientes[i];
17         if (valor >= 10) valor -= 9;
18         suma += valor;
19     }
20     return suma % 10 == 0 ? cedula : 'Cédula no válida';
21 }

```

```

18 }
19 final verificador = (10 - (suma % 10)) % 10;
20 if (verificador != int.parse(cedula[9])) return 'Cédula_no_válida';
21 return null;
22 }

```

Listing 9: Validador de cédula ecuatoriana

Algoritmo 2 Validación de cédula ecuatoriana (módulo 10)**Require:** Cadena c de 10 dígitos**Ensure:** *true* si la cédula es válida

```

1: if  $|c| \neq 10$  or  $c$  no es numérica then
2:   return false
3: end if
4:  $p \leftarrow \text{entero}(c[0,1])$  ▷ código de provincia
5: if  $p < 1$  or  $p > 24$  then
6:   return false
7: end if
8: if  $c[2] > 6$  then
9:   return false
10: end if
11:  $C \leftarrow [2, 1, 2, 1, 2, 1, 2, 1, 2]; \quad s \leftarrow 0$ 
12: for  $i \leftarrow 0$  to 8 do
13:    $v \leftarrow c[i] \times C[i]$ 
14:   if  $v \geq 10$  then
15:      $v \leftarrow v - 9$ 
16:   end if
17:    $s \leftarrow s + v$ 
18: end for
19:  $d \leftarrow (10 - (s \bmod 10)) \bmod 10$ 
20: return  $d = c[9]$ 

```

6.2.8 Polígono de finca y cálculo de área

El agricultor traza el contorno real de su finca tocando los vértices sobre el mapa. El área se calcula con la fórmula de área esférica sobre el elipsoide terrestre, y el centro del polígono (centroide) se usa para ubicar el marcador y centrar la cámara del mapa (Listado 10).

```

1  /// Área en hectáreas de un polígono definido por sus vértices (en
   orden).
2  static double areaHectareas(List<GeoPunto> puntos) {
3    if (puntos.length < 3) return 0;
4    const radioTierra = 6378137.0; // metros
5    double suma = 0;
6    for (var i = 0; i < puntos.length; i++) {
7      final p1 = puntos[i];
8      final p2 = puntos[(i + 1) % puntos.length];
9      suma += _rad(p2.lng - p1.lng) *
10             (2 + math.sin(_rad(p1.lat)) + math.sin(_rad(p2.lat)));
11    }

```

```

12 final areaM2 = (suma * radioTierra * radioTierra / 2).abs();
13 return areaM2 / 10000; // m2 -> ha
14 }

```

Listing 10: Área de un polígono geográfico en hectáreas

6.2.9 Gestión de permisos con aviso previo

En lugar de disparar el diálogo nativo del sistema sin contexto, la aplicación muestra primero una explicación del uso que dará al permiso, y maneja explícitamente el caso de *denegado permanentemente* ofreciendo abrir los ajustes de la app (Listado 11).

```

1 Future<bool> solicitarPermisoConAviso(
2   BuildContext context, {
3     required Permission permiso,
4     required String titulo,
5     required String mensaje,
6     required IconData icono,
7   }) async {
8     final estadoActual = await permiso.status;
9     if (estadoActual.isGranted) return true;
10
11     if (estadoActual.isPermanentlyDenied) {
12       final abrirAjustes = await _mostrarDialogo(context, /* ... */);
13       if (abrirAjustes) await openAppSettings();
14       return false;
15     }
16
17     final continuar = await _mostrarDialogo(context, /* ... */);
18     if (!continuar) return false;
19
20     final resultado = await permiso.request();
21     return resultado.isGranted;
22 }

```

Listing 11: Solicitud de permiso con aviso previo

6.2.10 Protección de claves de API

La clave de Google Maps no se escribe en el `AndroidManifest.xml`: se declara como `manifestPlaceholder` y Gradle la inyecta en tiempo de compilación leyéndola de `android/secrets.properties`, archivo excluido del control de versiones. En el manifiesto solo aparece la referencia `${MAPS_API_KEY}`. La clave de OpenWeather reside en `lib/core/constants/api_keys.dart`, también excluido del repositorio y distribuido como plantilla `.example`.

7. Panel Web Administrativo

El panel web es una aplicación Flutter Web independiente (`web_admin/`), desplegada en Firebase Hosting, que comparte el mismo proyecto de Firebase que la app móvil pero **no comparte código** con ella. Se decidió deliberadamente no implementar el rol de administrador dentro de la aplicación móvil: separar el binario evita enviar a los agricultores código y permisos que no les corresponden.

7.1 Control de acceso

La autenticación se realiza con Firebase Authentication (correo/contraseña o Google mediante *popup*), pero autenticarse **no basta**: tras validar la credencial, el sistema verifica que el UID exista como documento en la colección `admins`. Si no existe, se cierra la sesión inmediatamente y se informa que la cuenta carece de permisos. Un agricultor con cuenta válida no puede, por tanto, ingresar al panel (Listado 12). Los documentos de `admins` solo pueden crearse manualmente desde la consola de Firebase: las reglas de seguridad prohíben toda escritura desde el cliente.

```

1 Future<bool> login(String email, String password) async {
2   try {
3     final credential = await _auth.signInWithEmailAndPassword(
4       email: email, password: password,
5     );
6     final uid = credential.user!.uid;
7
8     final esAdmin = await _adminRepository.esAdmin(uid); // ¿existe
9                       /admins/{uid}?
10    if (!esAdmin) {
11      await _auth.signOut();
12      errorMessage = 'Esta cuenta no tiene permisos de administrador.';
13      return false;
14    }
15
16    usuario = credential.user;
17    nombreAdmin = usuario?.email?.split('@').first ?? 'Administrador';
18    return true;
19  } on FirebaseAuthException catch (e) {
20    errorMessage = switch (e.code) {
21      'user-not-found' => 'No existe una cuenta con ese correo.',
22      'wrong-password' || 'invalid-credential' => 'Correo o contraseña
23        incorrectos.',
24      'invalid-email' => 'El correo no es válido.',
25      _ => 'Ocurrió un error al iniciar sesión.',
26    };
27    return false;
28  } finally {
29    isLoading = false;
30    notifyListeners();
31  }
32 }

```

Listing 12: Verificación de privilegio de administrador

7.2 Arquitectura del panel

Al tratarse de un panel de **solo lectura** orientado al monitoreo, no se replicó la separación completa *domain/data/presentation* de la app móvil: se aplicó una arquitectura MVVM simplificada de dos capas.

- **AdminRepository**: único punto de acceso a Firestore. Expone métodos de lectura para usuarios, fincas, parcelas, cultivos y `diagnosticos_hojas`, más la comprobación `esAdmin(uid)`.
- **AdminDataViewModel**: carga las cinco colecciones en paralelo con `Future.wait` una sola vez, y expone **proyecciones ya combinadas** (un *join* en memoria, dado

que Firestore no soporta *joins* nativos) para cada vista: diagnósticos enriquecidos con el nombre del agricultor y de la parcela, resumen por agricultor, KPIs, distribución por clase, serie temporal y alertas de zona.

- **AdminAuthViewModel**: sesión y verificación de privilegio.
- **AdminShell**: *layout* con barra lateral fija y conmutación de sección mediante `switch` sobre un `enum AdminSection`.

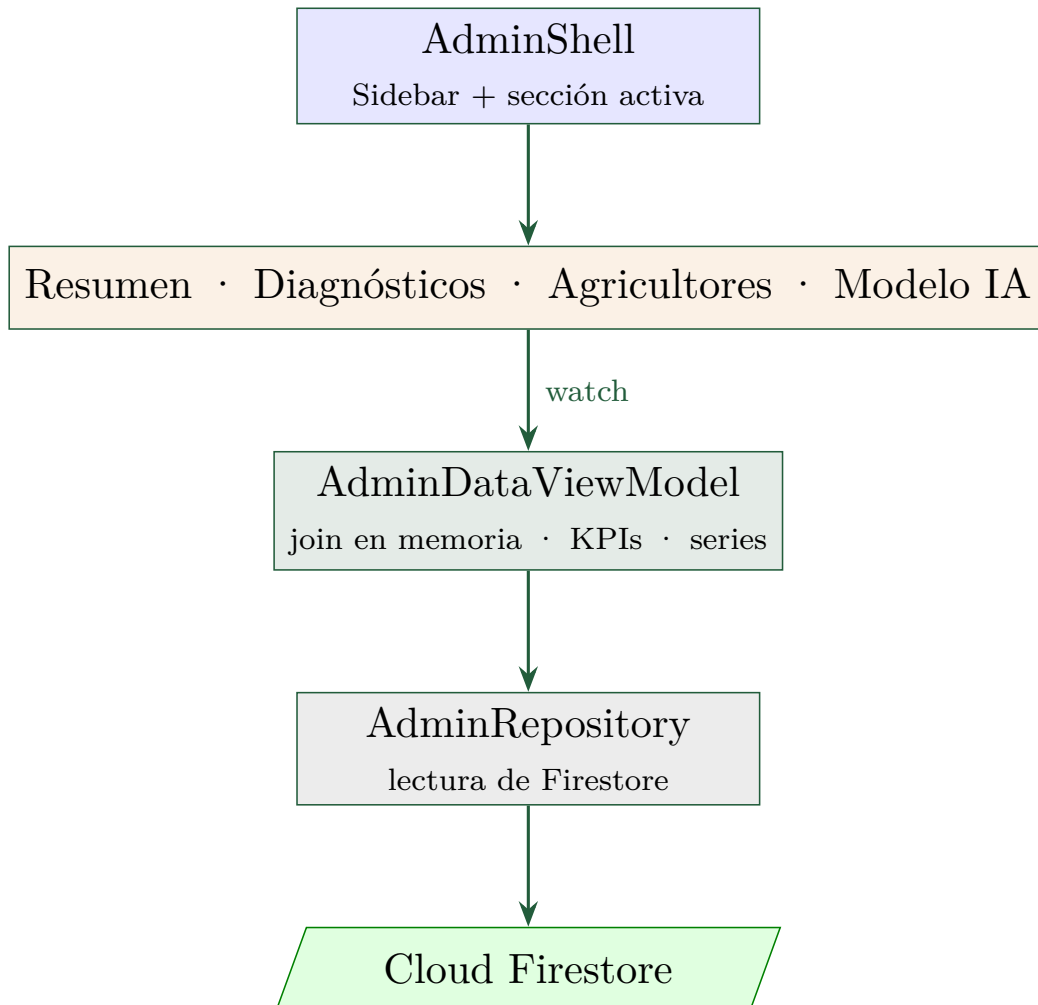


Figura 5: Arquitectura del panel web administrativo (MVVM simplificado)

7.3 Vistas del panel

7.3.1 Resumen (*dashboard*)

Concentra la lectura ejecutiva del estado fitosanitario:

- **Cuatro KPIs**: diagnósticos totales, porcentaje de hojas enfermas, agricultores activos y confianza media del modelo sobre los diagnósticos reales registrados.
- **Gráfico diario apilado** (`chartDiario`): conteo de hojas sanas frente a enfermas en los últimos 14 días.
- **Distribución por clase**: proporción de cada enfermedad sobre el total de diagnósticos.
- **Diagnósticos recientes**: los seis últimos, con agricultor, parcela, clase y confianza.

- **Alertas de zona** (alertasDeZona): parcelas con dos o más diagnósticos con daño de la misma enfermedad en los últimos 7 días, ordenadas por número de casos. Es el mecanismo de detección de un foco de propagación.

```

1  /// Parcelas con >= 2 diagnósticos con daño en los últimos 7 días.
2  List<({String parcela, String enfermedad, int casos})> alertasDeZona()
3  {
4      final limite = DateTime.now().subtract(const Duration(days: 7));
5      final conteo = <String, int>{};
6
7      for (final d in _diagnosticos) {
8          if (d.enfermedad == 'hoja_sana' || d.fecha.isBefore(limite))
9              continue;
10         final parcela = _nombreParcelaDeCultivo(d.cultivoId);
11         final clave = '$parcela|${d.enfermedad}';
12         conteo[clave] = (conteo[clave] ?? 0) + 1;
13     }
14
15     final resultado = conteo.entries
16         .where((e) => e.value >= 2)
17         .map((e) {
18             final partes = e.key.split('|');
19             return (parcela: partes[0], enfermedad: partes[1], casos: e.
20                 value);
21         })
22         .toList()
23         ..sort((a, b) => b.casos.compareTo(a.casos));
24
25     return resultado.take(3).toList();
26 }

```

Listing 13: Detección de focos de propagación (alertas de zona)

7.3.2 Diagnósticos

Tabla global de todos los diagnósticos del sistema, ordenados por fecha descendente y filtrables por clase de enfermedad mediante *chips*. Cada fila muestra agricultor, parcela, clase, confianza y fecha. Las filas con coordenadas GPS son **clicables**: abren la ubicación del diagnóstico en Google Maps en una pestaña nueva (`url_launcher`).

7.3.3 Agricultores

Listado de las cuentas registradas con su finca principal, número de parcelas, número de diagnósticos realizados y fecha del último diagnóstico. Permite identificar rápidamente a los usuarios activos y a los inactivos.

7.3.4 Modelo IA

Panel de rendimiento del clasificador: exactitud global, *recall* medio, número de clases entrenadas, tamaño del artefacto y, para cada clase, precisión y *recall* con una barra de progreso. Documenta también, de forma explícita, la clase *daño por plaga* como no soportada por falta de dataset.

7.4 Despliegue

El panel se compila con `flutter build web` y se publica en Firebase Hosting; la configuración de `firebase.json` apunta a `web_admin/build/web` con reescritura de todas las rutas a `index.html` (aplicación de página única).

8. Pruebas

Esta sección define el **plan de pruebas** del sistema. Los casos de prueba están especificados y listos para ejecutarse; la **ejecución formal con usuarios reales del cantón Sigchos se encuentra pendiente** y sus resultados se consignarán en la columna correspondiente y en el Anexo de evidencia.

8.1 Estrategia general

Se definen tres niveles de prueba:

1. **Pruebas de sistema (funcionales)**: verifican que cada requisito funcional se cumpla de extremo a extremo sobre el dispositivo físico, con datos reales de Firebase.
2. **Pruebas de usabilidad**: miden si un agricultor sin formación técnica logra completar las tareas centrales sin ayuda, y con qué grado de satisfacción.
3. **Pruebas de rendimiento**: miden tiempos de respuesta, consumo de memoria y comportamiento del clasificador bajo condiciones de campo (poca luz, sin conexión, gama baja).

Entorno de prueba: dispositivo físico Android real (TECNO KJ6, Android 13), conexión móvil intermitente, proyecto de Firebase en modo producción con reglas de seguridad activas.

8.2 Pruebas de usuario — usabilidad

8.2.1 Protocolo

- **Perfil de participantes**: 6 agricultores del cantón Sigchos, sin experiencia previa con la aplicación; se incluye al menos un participante mayor de 55 años y uno con baja alfabetización digital.
- **Método**: prueba de tareas con pensamiento en voz alta. El facilitador no interviene salvo bloqueo total.
- **Métricas**: tasa de éxito por tarea, tiempo hasta completarla, número de errores, número de solicitudes de ayuda.
- **Instrumento de satisfacción**: cuestionario SUS (System Usability Scale, 10 ítems, escala 1–5). Se establece como **criterio de aceptación** un puntaje SUS ≥ 68 , valor considerado el promedio de la industria.

Cuadro 6: Casos de prueba de usabilidad

ID	Tarea solicitada	Criterio de éxito	Tiempo objetivo	Resultado
PU-01	Crear una cuenta y entrar a la app	El usuario completa el registro sin ayuda; los errores de validación se entienden.	≤ 3 min	Pendiente
PU-02	Registrar su finca dibujando el contorno en el mapa	El polígono se cierra y el área en hectáreas aparece calculada.	≤ 4 min	Pendiente
PU-03	Diagnosticar una hoja con la cámara	Llega a la pantalla de resultado y sabe decir qué enfermedad detectó.	≤ 2 min	Pendiente
PU-04	Explicar qué debe hacer según la recomendación	Identifica al menos dos acciones concretas de la lista.	≤ 2 min	Pendiente
PU-05	Encontrar un diagnóstico anterior en el historial	Localiza el diagnóstico usando el filtro por enfermedad.	$\leq 1,5$ min	Pendiente
PU-06	Interpretar la alerta de riesgo fúngico del clima	Explica con sus palabras que hay riesgo de hongos.	≤ 1 min	Pendiente
PU-07	Ubicar un foco de enfermedad en el mapa	Toca un marcador rojo y describe a qué corresponde.	≤ 2 min	Pendiente

8.2.2 Aspectos de usabilidad ya incorporados al diseño

Estas decisiones anticipan los hallazgos esperados de la prueba:

- **Aviso previo de permisos:** se explica el uso de cámara y GPS antes de mostrar el diálogo del sistema, evitando el rechazo por desconfianza.
- **Marco guía en la cámara:** orienta al usuario a encuadrar una sola hoja.
- **Mensaje de rechazo accionable:** cuando el modelo devuelve `no_hoja`, el mensaje no dice “error” sino qué hacer (“acerca la cámara a una sola hoja, con buena luz y fondo simple”).
- **Anillo de confianza y severidad:** el resultado no muestra solo un nombre técnico, sino un indicador visual de gravedad.
- **Mostrar/ocultar contraseña:** reduce errores de tipeo en teclados pequeños.

8.3 Pruebas de rendimiento — pruebas del sistema

Cuadro 7: Casos de prueba de rendimiento

ID	Escenario	Métrica y criterio de aceptación	Instrumento	Resultado
PR-01	Arranque en frío de la app	Tiempo hasta la pantalla de Inicio ≤ 3 s (incluye carga del modelo TFLite).	Cronómetro / <code>flutter run -profile</code>	Pendiente
PR-02	Inferencia del modelo sobre una foto	Latencia de <code>Interpreter.run()</code> $\leq 1,5$ s en gama baja.	Stopwatch en <code>TFLiteService</code>	Pendiente
PR-03	Flujo completo captura $\rightarrow$ resultado	Tiempo total ≤ 6 s con red 4G.	Cronómetro	Pendiente
PR-04	Diagnóstico sin conexión a internet	La clasificación se completa igual; el diagnóstico se guarda sin imagen.	Modo avión	Pendiente
PR-05	Subida a Storage con red inestable	Los 3 reintentos con <i>backoff</i> se ejecutan; si fallan, no se pierde el diagnóstico.	Simulación de corte de red	Pendiente
PR-06	Historial con 100+ diagnósticos	<i>Scroll</i> fluido, sin caída bajo 55 FPS.	Flutter DevTools	Pendiente
PR-07	Mapa con 50 marcadores y polígonos	Renderizado ≤ 2 s, gestos sin bloqueo.	Flutter DevTools	Pendiente
PR-08	Consumo de memoria en el flujo de diagnóstico	Pico de memoria ≤ 250 MB.	DevTools <i>Memory</i>	Pendiente
PR-09	Carga del panel web con 500 diagnósticos	Primer render ≤ 4 s (5 colecciones en paralelo).	Chrome DevTools	Pendiente
PR-10	Tamaño del APK de <i>release</i>	≤ 60 MB.	<code>flutter build apk -release</code>	Pendiente

8.4 Casos de prueba funcionales (pruebas de sistema)

Cuadro 8: Casos de prueba funcionales del sistema

ID	Caso de prueba	Datos / pasos de entrada	Resultado esperado
CP-01	Registro con datos válidos	Nombre, cédula 1712345678, celular 0991234567, correo y contraseña válidos.	Cuenta creada; documento en usuarios ; navega a Inicio.

(continuación de la Tabla 8)

ID	Caso de prueba	Datos / pasos de entrada	Resultado esperado
CP-02	Registro con cédula inválida	Cédula 1712345679 (dígito verificador incorrecto).	Se muestra “Cédula no válida”; no se crea la cuenta.
CP-03	Registro con celular inválido	Celular 0891234567 (no empieza en 09).	Se muestra el error de formato; no se crea la cuenta.
CP-04	Login con credenciales incorrectas	Contraseña errónea.	Mensaje “Correo o contraseña incorrectos”; permanece en Login.
CP-05	Login con Google	Cuenta de Google válida.	Sesión iniciada; navega a Inicio.
CP-06	Sesión persistente	Cerrar y reabrir la app con sesión activa.	Entra directo a Inicio sin pedir credenciales.
CP-07	Registrar finca con polígono	Tocar 4 vértices en el mapa y guardar.	Finca creada con límite de 4 puntos y área en hectáreas > 0 .
CP-08	Registrar parcela y cultivo	Parcela bajo la finca; cultivo bajo la parcela.	Jerarquía finca $\rightarrow$ parcela $\rightarrow$ cultivo persistida en Firestore.
CP-09	Denegar permiso de cámara	Rechazar el permiso en el diálogo del sistema.	La app no crashea; explica por qué lo necesita y permite reintentar.
CP-10	Captura desde cámara en vivo	Fotografiar una hoja de zapallo enferma.	Se obtiene una clase distinta de <code>hoja_sana</code> con confianza $\geq 0,50$.
CP-11	Captura desde galería	Seleccionar imagen del dataset.	Mismo flujo de clasificación que la cámara.
CP-12	Rechazo de imagen no-hoja	Fotografiar un objeto (p. ej. una taza).	Se rechaza la imagen con mensaje accionable; no se guarda diagnóstico.
CP-13	Geoetiquetado del diagnóstico	GPS activo durante la captura.	El documento incluye <code>ubicacion</code> (GeoPoint) y el mini-mapa muestra el punto.
CP-14	Snapshot climático	Clima cargado antes del diagnóstico.	El documento incluye temperatura, humedad y descripción.

(continuación de la Tabla 8)

ID	Caso de prueba	Datos / pasos de entrada	Resultado esperado
CP-15	Recomendaciones por enfermedad	Diagnóstico de mildiu.	Se listan las recomendaciones de mildiú ordenadas por orden.
CP-16	Recomendaciones sin Firestore poblado	Colección recomendaciones vacía.	Se muestra el contenido de respaldo local; la pantalla nunca queda vacía.
CP-17	Historial filtrado	Filtrar por “Oídio”.	Solo se listan los diagnósticos de esa clase.
CP-18	Eliminar diagnóstico	Eliminar un ítem del historial.	Se borra el documento y también la imagen en Storage.
CP-19	Mapa de incidencia	Abrir el mapa con diagnósticos geolocalizados.	Se pintan el polígono de la finca y los marcadores por severidad; los filtros con/sin daño funcionan.
CP-20	Clima y pronóstico	Abrir la pantalla de Clima con GPS activo.	Se muestran clima actual y pronóstico de 5 días de la ubicación real.
CP-21	Alerta de riesgo fúngico	Humedad ≥ 80 % y temperatura entre 15–25 °C.	Se muestra la alerta y se dispara una notificación local (una vez por sesión).
CP-22	Notificación de resultado	Diagnóstico con enfermedad detectada.	Llega una notificación local con la clase y el porcentaje de confianza.
CP-23	Editar perfil con foto	Cambiar nombre y subir foto.	Foto en Storage (<code>perfiles/{uid}/avatar.jpg</code>) y <code>fotoUrl</code> actualizado.
CP-24	Seguridad: leer datos ajenos	Consultar un documento con <code>usuarioId</code> de otro usuario.	Firestore responde <code>permission-denied</code> .
CP-25	Seguridad: acceso al panel web sin ser admin	Login en el panel con cuenta de agricultor.	Se cierra la sesión y se informa que la cuenta no tiene permisos.
CP-26	Panel web: KPIs	Abrir Resumen con datos existentes.	Los KPIs coinciden con el conteo real de Firestore.

(continuación de la Tabla 8)

ID	Caso de prueba	Datos / pasos de entrada	Resultado esperado
CP-27	Panel web: alerta de zona	Dos diagnósticos con daño en la misma parcela en 7 días.	La parcela aparece en la lista de alertas de zona.
CP-28	Panel web: abrir ubicación	Clic en una fila de diagnóstico con GPS.	Se abre Google Maps en el punto exacto.
CP-29	Panel web: filtro de diagnósticos	Filtrar por una clase.	Solo se muestran los diagnósticos de esa clase.
CP-30	Modo simulado del clasificador	Eliminar <code>model.tflite</code> de los <i>assets</i> .	La app no crashea: cae a modo simulado y el flujo sigue siendo demostrable.

8.5 Evidencia de pruebas

La evidencia de la ejecución de los casos anteriores se recopilará en:

1. Capturas de pantalla de cada caso ejecutado (Anexo, imágenes `img/25` a `img/31`).
2. Registro de la consola de Firestore mostrando los documentos creados por los casos CP-01, CP-07, CP-08 y CP-13.
3. Salida de `flutter analyze` (análisis estático sin advertencias en ambos proyectos).
4. Salida de `flutter build apk` (compilación exitosa).
5. Tabla de resultados del cuestionario SUS con el puntaje promedio de los seis participantes.
6. Capturas de Flutter DevTools con las mediciones de FPS, memoria y latencia de inferencia.

9. Diagrama de Carpetas

El repositorio se organiza en dos proyectos Flutter independientes que comparten el mismo backend de Firebase.

PROYECTO FINAL/

```

firebase.json          <- hosting + apunta a firebase/
README.md
firebase/
  firestore.rules       <- reglas de seguridad de Firestore
  firestore.indexes.json <- indices compuestos
  storage.rules        <- reglas de Storage
docs/                  <- GitHub Pages (Play Store)
  index.html
  privacy/index.html   <- politica de privacidad
  eliminar-cuenta/index.html <- eliminacion de cuenta
mobile_app/           <- App Android (Flutter)
  android/
    app/src/main/AndroidManifest.xml

```



```

    secrets.properties.example    <- plantilla de la API key de Maps
assets/
  images/logo.png
  ml/model.tflite                <- modelo TFLite (float16, 4.3 MB)
  ml/labels.txt                  <- 6 etiquetas (5 clases + no_hoja)
lib/
  main.dart                      <- composicion de dependencias + MultiProvider
  app.dart                       <- MaterialApp + rutas
  firebase_options.dart
  core/
    constants/                   <- firestore_paths, enfermedades, api_keys
    errors/                      <- Failure y subclases
    models/                      <- GeoPunto
    routes/                      <- AppRoutes + AppRouter (transiciones)
    theme/                       <- AppColors, AppTheme
    utils/                       <- Validators, GeoUtils, permission_helper
    widgets/                     <- AppTextField, AppPrimaryButton, MainTabBar...
  features/
    auth/                        domain | data | presentation
    fincas/                      domain | data | presentation
    parcelas/                    domain | data | presentation
    cultivos/                    domain | data | presentation
    diagnostico/                 domain | data | presentation
    clima/                      domain | data | presentation
    recomendaciones/             domain | data | presentation
    historial/                   presentation (reusa el domain de diagnostico)
    mapa/                       presentation (reusa domain de fincas y diagnostico)
    inicio/                     presentation
  services/                      <- TFLiteService, LocationService,
                                StorageService, NotificationService
  tool/
    entrenar_modelo_colab.ipynb  <- notebook v1 (5 clases)
    entrenar_modelo_colab_v2.ipynb <- notebook v2 (+ clase no_hoja)
    seed_recomendaciones.dart     <- poblado de Firestore
web_admin/                       <- Panel administrativo (Flutter Web)
lib/
  main.dart  app.dart
  core/
    constants/                   <- firestore_paths, enfermedades
    models/                      <- UsuarioDoc, FincaDoc, DiagnosticoDoc...
    services/                    <- AdminRepository
    theme/                       <- AppColors, AppTheme
    viewmodels/                  <- AdminDataViewModel
    widgets/                     <- AdminSidebar
  features/
    auth/                        <- AdminAuthViewModel + AdminLoginView
    shell/                       <- AdminShell (layout + navegacion)
    dashboard/                   <- Resumen (KPIs, graficos, alertas de zona)
    diagnosticos/                 <- tabla global filtrable
    agricultores/                 <- listado de cuentas
    modelo_ia/                   <- metricas del clasificador

```

Estructura interna de cada *feature* de la app móvil:

```
features/<nombre>/
  domain/
    entities/      <- clases planas, sin Flutter ni Firebase
    repositories/  <- contratos abstractos
    usecases/      <- 1 clase = 1 accion de negocio
  data/
    models/        <- mapeo desde/hacia Firestore (fromMap / toMap)
    datasources/   <- acceso concreto (Firestore, HTTP, TFLite)
    repositories/  <- implementacion del contrato del domain
  presentation/
    viewmodels/    <- ChangeNotifier (estado + orquestacion)
    views/         <- pantallas
    widgets/       <- componentes propios de la feature
```

10. Modelo de Datos

La persistencia utiliza **Cloud Firestore**, una base de datos NoSQL orientada a documentos. Al no existir *joins* nativos, las relaciones se modelan mediante referencias por identificador y, cuando el costo de una consulta adicional no se justifica, mediante **desnormalización controlada** (por ejemplo, el `climaSnapshot` embebido dentro del documento del diagnóstico).

10.1 Colecciones

Cuadro 9: Colecciones de Firestore

Colección	Campos principales
usuarios	nombre, cedula, telefono, email, fotoUrl, rol, fechaRegistro
fincas	usuarioId, nombre, ubicacion (GeoPoint), limite (List<GeoPoint>), direccion, areaHectareas, fechaCreacion
parcelas	finsaId, usuarioId, nombre, areaHectareas, ubicacion (GeoPoint), fechaCreacion
cultivos	parcelaId, usuarioId, variedad, fechaSiembra, plantasEstimadas, etapa
diagnosticos_hojas	usuarioId, cultivoId, imagenUrl, enfermedad, confianza, top3[], ubicacion (GeoPoint), climaSnapshot, fecha
recomendaciones	enfermedad, orden, titulo, descripcion
admins	documento cuyo ID es el UID; su sola existencia otorga permiso de administrador
clima* / alertas*	reservadas para historial climático y notificaciones basadas en servidor

* Colecciones declaradas en el esquema y contempladas en las reglas de seguridad, pero aún no pobladas activamente (véase Recomendaciones).

10.2 Relaciones

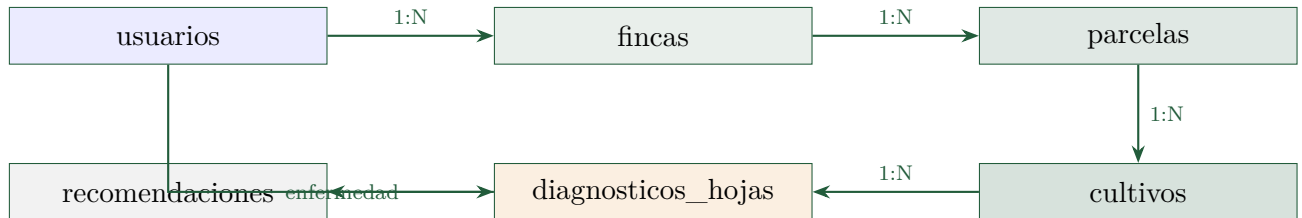


Figura 6: Relaciones lógicas entre colecciones (referencias por ID, sin *join* nativo)

10.3 Índices compuestos

Firestore exige un índice compuesto para toda consulta que combine **where** con **orderBy** sobre campos distintos. Se declararon cuatro en `firestore.indexes.json`:

Cuadro 10: Índices compuestos declarados

Colección	Campos del índice
fincas	usuarioId (asc) + fechaCreacion (desc)
parcelas	fincaId (asc) + fechaCreacion (desc)
diagnosticos_hojas	usuarioId (asc) + fecha (desc)
recomendaciones	enfermedad (asc) + orden (asc)

10.4 Reglas de seguridad

Las reglas exigen autenticación y verifican la **propiedad del documento** (`usuarioId == request.auth.uid`) o la pertenencia a la colección `admins` para la lectura ampliada del panel. La colección `recomendaciones` es de lectura para cualquier usuario autenticado y de escritura solo para administradores; la colección `admins` no admite escritura desde ningún cliente (Listado 14).

```

1 rules_version = '2';
2 service cloud.firestore {
3   match /databases/{database}/documents {
4
5     function esAdmin() {
6       return request.auth != null &&
7         exists(/databases/{database}/documents/admins/{request.auth.uid});
8     }
9
10    function esDueno(datos) {
11      return request.auth != null && datos.usuarioId == request.auth.uid;
12    }
13  }

```

```

14     match /diagnosticos_hojas/{doc} {
15         allow read:          if esDueno(resource.data) || esAdmin();
16         allow create:        if esDueno(request.resource.data);
17         allow update, delete: if esDueno(resource.data);
18     }
19
20     match /recomendaciones/{doc} {
21         allow read:  if request.auth != null;
22         allow write: if esAdmin();
23     }
24
25     match /admins/{uid} {
26         allow read:  if request.auth != null && request.auth.uid == uid;
27         allow write: if false; // solo desde la consola de Firebase
28     }
29 }
30 }

```

Listing 14: Reglas de seguridad de Cloud Firestore (extracto)

Las reglas de Storage, además de exigir que el usuario sea el dueño de la ruta, restringen el **tipo de contenido** a imágenes y el **tamaño** a 10 MB por archivo:

```

1 match /diagnosticos/{usuarioId}/{archivo} {
2     allow read:  if request.auth != null;
3     allow write: if request.auth != null && request.auth.uid ==
4         usuarioId
5         && request.resource.size < 10 * 1024 * 1024
6         && request.resource.contentType.matches('image/*');
7 }

```

Listing 15: Reglas de Firebase Storage (extracto)

11. Trabajos Relacionados

Según Hasan et al. (2024), en su artículo “*Automated Disease Diagnosis in Pumpkin Plants Using Advanced CNN Models*” [1], mencionan que evaluaron los modelos ResNet-34, DenseNet-121 y EfficientNet-B7 sobre 2000 imágenes de hojas de zapallo distribuidas en cinco clases —incluyendo mildiú vellosa y oídio—, reportando que DenseNet-121 alcanzó la mayor exactitud (86 %), un valor de referencia frente al 83 % obtenido en este proyecto con una arquitectura deliberadamente más liviana. De manera similar, según los autores del estudio “*Explainable AI-Enhanced Deep Learning for Pumpkin Leaf Disease Detection*” (2025) [2], mencionan que emplearon MobileNetV3Large adaptado para dispositivos móviles junto con técnicas de IA explicable, siendo pioneros en aplicar dichas técnicas a la clasificación de enfermedades de zapallo, lo que respalda directamente la elección de una arquitectura de la familia MobileNet para la inferencia *on-device* implementada en Sigchos Agrotech.

Según Santoso et al. (2024), en su artículo “*Enhancing Apple Leaf Disease Detection with Deep Learning: From Model Training to Android App Integration*” [3], mencionan que integraron el modelo ReXNet-150 en una aplicación Android mediante TensorFlow Lite para lograr un diagnóstico en tiempo real sin conexión a internet, validando la viabilidad técnica del enfoque adoptado en este proyecto. Asimismo, según el estudio “*A Mobile-Based System for Detecting Plant Leaf Diseases Using Deep Learning*” [4], mencionan que un modelo convertido al formato `.tflite` permitió la detección, segmentación y

seguimiento de enfermedades foliares del maíz directamente desde el dispositivo móvil, con tiempos de respuesta compatibles con el uso en campo. Complementariamente, según Aarif et al. (2025) [6], en su revisión sobre tecnologías de sensores para agricultura de precisión, mencionan que la integración de sensores GPS y datos climáticos en tiempo real habilita alertas tempranas de riesgo fitosanitario, principio que Sigchos Agrotech aplica en su módulo de clima y geolocalización; y según el trabajo de clasificación con CNN y transferencia de aprendizaje publicado en *Electronics* [5], mencionan que dicha técnica supera consistentemente al entrenamiento desde cero cuando el dataset disponible es de tamaño moderado, justificando la estrategia de entrenamiento seguida en este proyecto.

12. Avance Técnico

12.1 Arquitectura aplicada

La arquitectura descrita en la Sección 4 se aplicó de forma consistente en las diez *features* de la app móvil. La composición de dependencias es manual (sin *service locator*) y se centraliza en `main.dart`: cada `DataSource` se envuelve en su `RepositoryImpl`, este se inyecta en los `UseCase`, y estos en el `ViewModel` correspondiente, registrado en un `MultiProvider` único.

Dos *features* ilustran la economía del diseño: `historial` y `mapa` **no tienen capa *domain* ni *data* propias**; reutilizan los casos de uso de `diagnostico` y `fincas`. El `MapaViewModel`, por ejemplo, se suscribe a los *streams* de `ObtenerFincasUseCase` y `ObtenerHistorialUseCase` y solo combina sus resultados, evitando duplicar el acceso a datos.

12.2 Estructura del proyecto

Véase la Sección 9 (Diagrama de Carpetas). Adicionalmente, el enrutamiento se centraliza en `core/routes/app_router.dart`, que resuelve cada ruta nombrada con una transición personalizada (*fade* + *slide* mediante `PageRouteBuilder`), evitando la duplicación de `Navigator.push` con `MaterialPageRoute` a lo largo de la aplicación. Las 16 rutas se declaran como constantes en `AppRoutes`.

12.3 Pantallas desarrolladas

Cuadro 11: Pantallas implementadas en la app móvil (mínimo requerido: 4)

Pantalla	Descripción funcional
Splash / Carga	Splash nativo a pantalla completa; resuelve la sesión activa antes del primer <i>frame</i> y salta directo a Inicio o Login.
Login	Correo-contraseña y Google Sign-In; campo de contraseña con mostrar/ocultar.
Registro	Nombre, cédula, celular, correo y contraseña, con validación en tiempo real.
Inicio	Saludo con avatar, clima resumido, finca activa, botón de diagnóstico, últimos resultados y alerta de zona.
Captura	Cámara en vivo embebida con marco guía; alternativa de galería.
Analizando	Estado de inferencia del modelo TFLite.
Resultado del diagnóstico	Imagen, severidad, anillo de confianza, top-3 de probabilidades, mini-mapa satelital y definición de la enfermedad.
Recomendaciones	Lista numerada de acciones agronómicas según la enfermedad detectada.
Historial	Lista filtrable por clase, con eliminación.
Mapa de zonas afectadas	Satelital, polígono de la finca, filtros con/sin daño, navegación a foco, leyenda de severidad y botón “mi ubicación”.
Clima	Clima actual, alerta de riesgo fúngico y pronóstico de 5 días.
Fincas	Listado jerárquico de fincas del agricultor.
Registrar finca	Mapa interactivo para trazar el polígono (deshacer/-limpiar), área calculada automáticamente.
Registrar parcela	Alta de parcela bajo una finca, con GPS.
Registrar cultivo	Variedad, fecha de siembra, plantas estimadas y etapa.
Perfil / Editar perfil	Datos del agricultor y foto, con subida a Storage.

Cuadro 12: Pantallas implementadas en el panel web administrativo

Pantalla	Descripción funcional
Login de administrador	Correo-contraseña o Google; rechaza cuentas que no estén en admins .
Resumen	4 KPIs, gráfico diario de 14 días, distribución por clase, diagnósticos recientes y alertas de zona.
Diagnósticos	Tabla global filtrable por clase; las filas con GPS abren Google Maps.
Agricultores	Cuentas con finca, número de parcelas, diagnósticos y último diagnóstico.
Modelo IA	Exactitud, <i>recall</i> medio, clases entrenadas, tamaño y métricas por clase.

12.4 Código principal explicado

La Sección 6.2 documenta, con su explicación, los listados centrales del sistema: la entidad de dominio (Listado 1), el caso de uso (Listado 2), la composición de dependencias (Listado 3), el ViewModel del diagnóstico (Listado 4), el servicio de inferencia TFLite (Listados 5 y 6), la subida tolerante a fallo (Listado 7), la regla de riesgo fúngico (Listado 8), la validación de cédula (Listado 9), el cálculo de área (Listado 10) y la gestión de permisos (Listado 11). La Sección 7 documenta el código del panel web (Listados 12 y 13) y la Sección 10 las reglas de seguridad (Listados 14 y 15).

12.5 Base de datos utilizada

Cloud Firestore (Google Firebase) en modo nativo. Se emplean cuatro índices compuestos para las consultas que combinan **where** con **orderBy** (Tabla 10). El almacenamiento de imágenes usa **Firebase Storage** bajo el plan Blaze (dentro de la capa gratuita de 5 GB), con rutas segregadas por usuario: `diagnosticos/{uid}/{timestamp}.jpg` y `perfiles/{uid}/avatar.jpg`.

12.6 Evidencia de funcionamiento

El funcionamiento se verificó mediante:

1. `flutter analyze` sin advertencias en `mobile_app` y `web_admin`.
2. Compilación exitosa del APK (`flutter build apk`).
3. Instalación y ejecución en **dispositivo físico Android real** (TECNO KJ6, Android 13) mediante *USB debugging*, cubriendo el flujo completo: registro/login, alta de finca con polígono, captura de hoja, clasificación, guardado en Firestore/Storage, historial, mapa y clima con datos reales.
4. Verificación de los documentos generados en la consola de Firestore y de las imágenes en Storage.
5. Despliegue del panel web en Firebase Hosting y comprobación del control de acceso con una cuenta sin privilegios.
6. Distribución de la aplicación en el canal de **prueba interna** de Google Play Console, con la ficha de la tienda, la política de privacidad y la página de

eliminación de cuenta publicadas en GitHub Pages.
Las capturas de esta evidencia se incluyen en los Anexos.

12.7 Problemas encontrados

- **Doble normalización de la imagen (el error más costoso):** el modelo exportado incluye `preprocess_input` de MobileNetV2 como capa interna, pero la primera versión del `TFLiteService` también normalizaba los píxeles en Dart. La transformación se aplicaba dos veces y toda predicción caía en la clase `no_hoja`. Se resolvió pasando los píxeles crudos `[0, 255]` al intérprete.
- **Falsos positivos con imágenes que no son hojas:** el modelo v1 (5 clases) asignaba con alta confianza una enfermedad a cualquier fotografía, incluidos objetos y fondos. Se resolvió entrenando un modelo v2 con una sexta clase de rechazo `no_hoja`, construida con hojas de otras especies y objetos, más un umbral mínimo de confianza.
- **Firestore Storage exige plan Blaze:** los proyectos nuevos de Firebase requieren facturación habilitada para usar Storage, aun dentro de la capa gratuita. Se activó Blaze y, mientras tanto, se implementó la subida de imágenes tolerante a fallo para no bloquear el diagnóstico.
- **Emulador de Android inestable:** el emulador de Android Studio provocó inestabilidad del sistema operativo en el equipo de desarrollo; se optó por realizar todas las pruebas en dispositivo físico vía *USB debugging*.
- **Discrepancia en los nombres de clase del dataset:** los nombres reales de las carpetas del dataset de Kaggle (`Healthy Leaf`, no `Healthy`) no coincidían con el mapeo inicial. Se agregó normalización de texto y un listado de carpetas disponibles en el notebook para depurar el mapeo.
- **Clase “daño por plaga” sin dataset:** no existe un conjunto público etiquetado adecuado. La clase se retiró del catálogo, de los colores y de las recomendaciones de ambas aplicaciones, en lugar de mantenerla como una clase que el clasificador nunca podría predecir.
- **Permisos solicitados sin aviso previo:** la primera versión pedía cámara y GPS directamente con el diálogo nativo. Se implementó el diálogo de aviso previo reutilizable `solicitarPermisoConAviso`, que explica el uso antes de disparar el permiso del sistema y gestiona el caso de denegación permanente.
- **Core library desugaring:** `flutter_local_notifications` requirió habilitar el *desugaring* de Java 8 en Gradle, no configurado por defecto en un proyecto Flutter nuevo.
- **Splash con franja visible:** el splash de Flutter se renderizaba con métricas de ventana incompletas. Se resolvió preservando el splash **nativo** (`flutter_native_splash`) hasta que la app dibuja su primer *frame* real, y resolviendo la sesión activa por debajo de él.
- **Fincas representadas por un solo punto:** un pin no comunica la extensión real de la finca. Se implementó el trazado de polígonos con cálculo de área esférica, manteniendo compatibilidad con las fincas antiguas que solo guardaron un punto GPS.

12.8 Mejoras pendientes

- Recolectar y etiquetar un dataset propio de “daño por plaga” para reincorporar la clase al modelo.
- Ampliar los datos de `mancha_foliar`, la clase con menor precisión (0,66), confundida con mildiú y oídio.
- Actualizar la vista “Modelo IA” del panel web con las métricas del modelo v2 y el tamaño real del artefacto (4,3 MB); actualmente muestra las métricas del v1.
- Ejecutar formalmente el plan de pruebas de la Sección 8 con agricultores reales y consignar los resultados.
- Publicar la aplicación en Google Play Store en producción (cuenta en proceso de verificación).
- Implementar el CRUD de recomendaciones y la emisión de alertas manuales desde el panel web administrativo.
- Poblar activamente las colecciones `clima` y `alertas` y migrar a notificaciones *push* reales basadas en servidor (Cloud Functions + FCM).
- Incorporar pruebas automatizadas: unitarias sobre la capa *domain* (con `mocktail`) y de *widget* sobre las vistas críticas.
- Habilitar un modo sin conexión con caché local para el historial y las recomendaciones.
- Migrar la clave de OpenWeather a un mecanismo de *secrets* equivalente al ya aplicado a la clave de Google Maps.

13. Conclusiones

Las conclusiones responden, en el mismo orden, a los ocho objetivos específicos planteados en la Sección 3.2.

1. **(Objetivo 1 — Arquitectura)** Se diseñó e implementó la aplicación móvil bajo **Clean Architecture**, con las capas *domain*, *data* y *presentation* y el patrón **MVVM** con **Provider**, en las diez *features* del proyecto. La regla de dependencia unidireccional se respetó: la capa *domain* no importa Flutter ni Firebase. El análisis estático (`flutter analyze`) no reporta advertencias en ninguno de los dos proyectos. Se cumple el primer objetivo específico.
2. **(Objetivo 2 — Modelo de IA)** Se entrenó un modelo por transferencia de aprendizaje sobre **MobileNetV2** en Google Colab (12 épocas de cabeza + 8 de *fine-tuning*), se exportó a **TensorFlow Lite** con cuantización `float16` (4,3 MB) y se integró en la aplicación con la normalización coherente con la del entrenamiento. El modelo alcanza un **83 % de exactitud** y un **83 % de *recall* macro** sobre 400 imágenes de validación, e incorpora una clase de rechazo `no_hoja` que impide diagnosticar fotografías que no corresponden a una hoja de zapallo. Se cumple el segundo objetivo específico, incluida la condición de rechazo.
3. **(Objetivo 3 — Firebase)** Se integraron **Firebase Authentication** (correo/-contraseña y Google Sign-In), **Cloud Firestore** (siete colecciones en uso activo, con cuatro índices compuestos) y **Firebase Storage** (con subida tolerante a fallo y reintentos con *backoff* exponencial). Las reglas de seguridad verifican la propiedad del documento y el privilegio de administrador. Se cumple el tercer objetivo específico.
4. **(Objetivo 4 — Clima)** Se consumió la **API REST de OpenWeather** para

obtener el clima actual y el pronóstico de cinco días a partir de la posición GPS, y se implementó la regla de alerta de riesgo fúngico (humedad $\geq 80\%$ y temperatura entre 15 y 25 °C), que además dispara una notificación local. El clima del momento queda embebido en cada diagnóstico, habilitando el análisis posterior de la correlación entre condiciones ambientales e incidencia. Se cumple el cuarto objetivo específico.

5. **(Objetivo 5 — Hardware móvil)** Se incorporaron la **cámara en vivo** (paquete **camera**, sin depender de la aplicación de cámara del sistema), el **GPS** (geolocator) y **Google Maps** en modo satelital con filtros, marcadores por severidad, navegación táctil a los focos y el contorno poligonal real de cada finca con su área calculada. Se cumple el quinto objetivo específico.
6. **(Objetivo 6 — Panel web)** Se desarrolló el **panel web administrativo** en Flutter Web como aplicación independiente, con control de acceso mediante la colección **admins** y cuatro vistas de monitoreo (Resumen, Diagnósticos, Agricultores y Modelo IA), incluyendo la detección automática de focos de propagación por parcela. La funcionalidad administrativa no se expone como un rol dentro de la aplicación móvil, tal como se propuso. Se cumple el sexto objetivo específico.
7. **(Objetivo 7 — Validaciones)** Se implementaron validaciones de cédula ecuatoriana mediante el algoritmo de módulo 10 (incluyendo la verificación del código de provincia y del tercer dígito), de número celular con prefijo 09 y 10 dígitos, de correo, de campos obligatorios, de valores numéricos positivos y de fechas no futuras, aplicadas en todos los formularios de la aplicación. Se cumple el séptimo objetivo específico.
8. **(Objetivo 8 — Pruebas)** Se definió y documentó el plan de pruebas con 30 casos funcionales (CP-01 a CP-30), 7 casos de usabilidad (PU-01 a PU-07) con el instrumento SUS y criterio de aceptación ≥ 68 , y 10 casos de rendimiento (PR-01 a PR-10) con métricas y criterios de aceptación explícitos. El objetivo se cumple en su componente de **definición**; su **ejecución formal con usuarios reales queda pendiente** y constituye la primera de las recomendaciones.

Conclusión general. El proyecto demuestra que es viable poner un diagnóstico fitosanitario asistido por inteligencia artificial en manos de un agricultor de una zona rural con conectividad limitada: la inferencia ocurre en el propio teléfono, en menos de dos segundos y sin necesidad de internet, y el contexto de geolocalización y clima transforma un diagnóstico individual en información de monitoreo zonal aprovechable por un técnico agrícola. La exactitud alcanzada (83 %) es adecuada para un **diagnóstico preliminar** que orienta la decisión del agricultor, pero no sustituye el criterio de un ingeniero agrónomo, y así se comunica dentro de la aplicación.

14. Recomendaciones

1. **Ejecutar el plan de pruebas** definido en la Sección 8 con los seis agricultores del perfil descrito, y ajustar la interfaz según los puntos de fricción que revele el SUS antes de cualquier despliegue masivo.
2. **Ampliar el dataset de mancha foliar**, que presenta la precisión más baja (0,66) por confusión con mildiú y oídio. Es la intervención con mayor impacto esperado sobre la exactitud global.
3. **Recolectar un dataset propio de “daño por plaga”** fotografiando hojas afectadas en las fincas del cantón, y reincorporar la clase al modelo. Esto convierte

una limitación del dataset público en una ventaja de contexto local.

4. **Reentrenar periódicamente el modelo** con las imágenes reales que los agricultores ya están subiendo a Storage, previa validación de un agrónomo. El sistema, tal como está construido, genera de forma natural el dataset que necesita para mejorar.
5. **Actualizar las métricas del panel web** para reflejar el modelo v2 en producción, de modo que el administrador no tome decisiones sobre datos del modelo anterior.
6. **Migrar a notificaciones *push* desde servidor** (Cloud Functions + FCM) poblando las colecciones *clima* y *alertas*, para que un agricultor reciba el aviso de riesgo fúngico aunque no haya abierto la aplicación.
7. **Incorporar pruebas automatizadas** sobre la capa *domain*, que al no depender de Flutter ni de Firebase es directamente comprobable con *mocks*; es el retorno más alto por esfuerzo invertido en calidad.
8. **Migrar la clave de OpenWeather** al mismo mecanismo de *secrets* ya aplicado a la clave de Google Maps antes de abrir el repositorio al público.
9. **Completar la verificación de la cuenta de Play Console** y publicar la aplicación en producción, dado que la ficha, la política de privacidad y la página de eliminación de cuenta ya están listas.
10. **Añadir un modo sin conexión con caché local**, coherente con la realidad de conectividad del cantón: el diagnóstico ya funciona sin internet, pero el historial y las recomendaciones aún no.

Referencias

- [1] Hasan, M. et al. (2024). *Automated Disease Diagnosis in Pumpkin Plants Using Advanced CNN Models*. arXiv:2410.00062. Recuperado de <https://arxiv.org/html/2410.00062v1>
- [2] Rahman, M. et al. (2025). *Explainable AI-Enhanced Deep Learning for Pumpkin Leaf Disease Detection: A Comparative Analysis of CNN Architectures*. arXiv:2501.05449. Recuperado de <https://arxiv.org/html/2501.05449v1>
- [3] Santoso, D. et al. (2024). *Enhancing Apple Leaf Disease Detection with Deep Learning: From Model Training to Android App Integration*. Journal of Applied Data Sciences. Recuperado de <https://bright-journal.org/Journal/index.php/JADS/article/view/507>
- [4] Ahmed, I. et al. (2023). *A Mobile-Based System for Detecting Plant Leaf Diseases Using Deep Learning*. MDPI AgriEngineering, 3(3), 32. Recuperado de <https://www.mdpi.com/2624-7402/3/3/32>
- [5] Chohan, M. et al. (2021). *Identification of Plant-Leaf Diseases Using CNN and Transfer-Learning Approach*. Electronics, MDPI, 10(12), 1388. Recuperado de <https://www.mdpi.com/2079-9292/10/12/1388>
- [6] Aarif, K. O. et al. (2025). *Smart Sensor Technologies Shaping the Future of Precision Agriculture: Recent Advances and Future Outlooks*. Journal of Sensors, Wiley. Recuperado de <https://onlinelibrary.wiley.com/doi/full/10.1155/js/2460098>
- [7] tahmidmir. (2023). *Pumpkin Leaf Diseases Dataset From Bangladesh*. Kaggle. Recuperado de <https://www.kaggle.com/datasets/tahmidmir/pumpkin-leaf-diseases-dataset-from-bangladesh>
- [8] Google LLC. (2024). *LiteRT (TensorFlow Lite) — On-device machine learning*. Recuperado de <https://ai.google.dev/edge/litert>
- [9] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. & Chen, L.-C. (2018). *MobileNetV2: Inverted Residuals and Linear Bottlenecks*. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). Recuperado de <https://arxiv.org/abs/1801.04381>
- [10] Google LLC. (2024). *Flutter — Build apps for any screen*. Recuperado de <https://flutter.dev/docs>
- [11] Rousselet, R. (2024). *Provider — State management for Flutter*. Recuperado de <https://pub.dev/packages/provider>
- [12] Google LLC. (2024). *Cloud Firestore Documentation*. Recuperado de <https://firebase.google.com/docs/firestore>
- [13] OpenWeather Ltd. (2024). *OpenWeather API Documentation*. Recuperado de <https://openweathermap.org/api>
- [14] Brooke, J. (1996). *SUS: A ‘quick and dirty’ usability scale*. En *Usability Evaluation in Industry*, Taylor & Francis, pp. 189–194.

Anexos

Inventario de capturas requeridas

Subir a Overleaf una carpeta **img/** con los siguientes archivos. Las rutas en este documento van **sin extensión**, por lo que funcionan tanto con **.png** como con **.jpg**.

Cuadro 13: Inventario de capturas para la carpeta **img/**

Archivo	Qué se debe capturar
logo	Logotipo de Sigchos Agrotech (portada).
<i>Aplicación móvil</i>	
01-splash	Pantalla de arranque con el logo.
02-login	Login, con el ícono de mostrar/ocultar contraseña visible.
03-registro	Registro mostrando un error de validación de cédula.
04-inicio	Inicio: saludo, clima, finca activa y últimos diagnósticos.
05-permiso-camara	Diálogo de aviso previo del permiso de cámara.
06-captura	Cámara en vivo con el marco guía sobre una hoja.
07-analizando	Pantalla de análisis (inferencia en curso).
08-resultado	Resultado: anillo de confianza, severidad, top-3 y mini-mapa.
09-rechazo-nohoja	Mensaje de rechazo al fotografiar algo que no es una hoja (CP-12).
10-recomendaciones	Lista de recomendaciones de la enfermedad detectada.
11-historial	Historial con el filtro por enfermedad aplicado.
12-mapa	Mapa satelital con el polígono de la finca y marcadores por severidad.
13-clima	Clima actual con la alerta de riesgo fúngico y el pronóstico.
14-fincas	Listado de fincas del agricultor.
15-registrar-finca	Trazado del polígono de la finca con el área calculada.
16-registrar-parcela	Alta de parcela con GPS.
17-registrar-cultivo	Alta de cultivo (variedad, siembra, etapa).
18-perfil	Perfil del agricultor con foto.
19-notificacion	Notificación local de resultado de diagnóstico.
<i>Panel web administrativo</i>	
20-admin-login	Login del panel (y, si es posible, el mensaje de acceso denegado a una cuenta no admin).
21-admin-resumen	Resumen: KPIs, gráfico diario, distribución y alertas de zona.
22-admin-diagnostico	Tabla de diagnósticos con un filtro aplicado.
23-admin-agricultores	Listado de agricultores.
24-admin-modelo	Vista Modelo IA con las métricas por clase.
<i>Evidencia técnica y de pruebas</i>	
25-flutter-analyze	Terminal con flutter analyze sin advertencias (ambos proyectos).

Archivo	Qué se debe capturar
26-build-apk	Terminal con la compilación exitosa del APK.
27-firestore	Consola de Firestore mostrando un documento de <code>diagnosticos_hojas</code> .
28-storage	Consola de Storage con las imágenes subidas.
29-colab-entrenamiento	Tabla de Colab con las curvas de entrenamiento.
30-matriz-confusion	Matriz de confusión / <code>classification_report</code> del modelo.
31-playstore	Play Console con la app en el canal de prueba interna.

A. Capturas de la aplicación móvil

Anexo 1: Pantalla de Login

Anexo 2: Registro con validación de cédula

Anexo 3: Pantalla de Inicio

Anexo 4: Aviso previo de permiso de cámara

Anexo 5: Captura de hoja con cámara en vivo

Anexo 6: Resultado del diagnóstico

Anexo 7: Rechazo de imagen que no es hoja (clase `no_hoja`)

Anexo 8: Recomendaciones agrícolas

Anexo 9: Historial de diagnósticos filtrable

Anexo 10: Mapa de zonas afectadas con polígono de finca

Anexo 11: Clima y alerta de riesgo fúngico

Anexo 12: Trazado del polígono de la finca con área calculada

Anexo 13: Perfil del agricultor

Anexo 14: Notificación local de resultado

B. Capturas del panel web administrativo

Anexo 15: Panel web administrativo — Resumen (KPIs, gráficos y alertas de zona)

Anexo 16: Panel web administrativo — Diagnósticos globales filtrables

Anexo 17: Panel web — Agricultores

Anexo 18: Panel web — Modelo IA

Anexo 19: Panel web — Login restringido a la colección `admins`

C. Evidencia de funcionamiento y de pruebas

Anexo 20: flutter analyze sin advertencias

Anexo 21: Compilación exitosa del APK

Anexo 22: Documento de diagnosticos_hojas en Firestore

Anexo 23: Imágenes de hojas en Firebase Storage

Anexo 24: Entrenamiento del modelo en Google Colab

Anexo 25: Matriz de confusión sobre el conjunto de validación

Anexo 26: Aplicación distribuida en el canal de prueba interna de Google Play